

නිපුණතාව 9 ගැටලු විසඳීමට ඇල්ගොරිතම සංවර්ධනය කර ඒවා ආකේතනය කිරීම සඳහා පයිතන් ක්‍රමලේඛ භාෂා භාවිතා කිරීම.

ගැටලු විසඳීමේ ක්‍රියාවලිය භාවිතා කිරීම.

පරිගණක ක්‍රමලේඛනයක් යනු කුමක්ද ? (What is a Computer Program?)

කිසියම් කාර්යයක් පරිගණකයෙන් කර ගැනීම සඳහා පරිගණක භාෂාවක් අනුසාරයෙන් ගොනුකළ ප්‍රකාශනයන්ගේ එකතුවක් පරිගණක ක්‍රමලේඛයක් (Computer Program) ලෙස හැඳින්වේ.

ක්‍රමලේඛන සංවර්ධන ජීවන චක්‍රය (Program Development Life-Cycle)



ගැටළුවක් විසඳීමේ අදියර (Analyzing the problem)

ගැටළුව විශ්ලේෂණය කිරීම

ගැටළුවක් විශ්ලේෂණය කිරීම යනු ගැටළුවට අදාළ ආදාන, ප්‍රතිදාන සහ ක්‍රියාවලිය හඳුනාගැනීමයි.



විකල්ප විසඳුම

ගැටලුවක් විසඳීම සඳහා ක්‍රමවේද කිහිපයක් තිබිය හැක. ඒවා විකල්ප විද්‍යාව ලෙස හඳුන්වයි. මෙය ගැටලුවේ ස්වභාවය අනුව වෙනස් වේ.

උදාහරණ: සාප්පකෝණාස්‍රයක පරිමිතිය සෙවීම සඳහා පහ ක්‍රමවේද භාවිතා කළ හැක.

පරිමිතිය=දිග+දිග+පළල+පළල

පරිමිතිය= $2 \times$ දිග + $2 \times$ පළල

පරිමිතිය=(දිග+ පළල) $\times$ 2

ගැටලු විසඳීම සඳහා ඇල්ගොරිතමික ප්‍රවේශය යොදාගැනීම.

ඇල්ගොරිතමක් යනු කුමක්ද? (What is an algorithm)

ඇල්ගොරිතමක් යනු ගැටළුවක් විසඳීම සඳහා අනුගමනය කරන ක්‍රමවේදය අනුපිළිවෙලින් ලියා දක්වයි.

මෙහිදී ගැටළුව විසඳීම සඳහා අනුගමනය කරන ක්‍රමවේදය අනුපිළිවෙලින් ලියා දක්වයි. ඇල්ගොරිතම ගොඩනැගීම රූපමය ආකාරයෙන් (Graphically) හෝ ලිඛිත ආකාරයෙන් (Textual) දක්විය හැක. රූපමය ආකාරයෙන් දැක්වීමට ගැලීම් සටහන් ද ලිඛිත ආකාරයෙන් දැක්වීමට ව්‍යාජ කේත ද භාවිතා කළ හැක.

ක්‍රමලේඛ ලිවීම (Writing a Computer Program)

ඇල්ගොරිතම පරිගණක භාෂාවක් ආධාරයෙන් ලියා දක්වීම මෙහිදී සිදුකරයි. මෙය "Coding" ලෙස හඳුන්වයි.

- | | | |
|---|---|--------------------|
| <ol style="list-style-type: none">1. Java2. C#3. VB4. Python | } | Computer Languages |
|---|---|--------------------|

ක්‍රමලේඛ ක්‍රියාත්මක කර බැලීම

(Testing a Computer Program)

ලියන ලද ක්‍රමලේඛය නිවැරදි දැයි මෙහිදී පරීක්ෂා කරනු ලබයි. වැරදි (Bugs) තිබෙනම් ඒවා "Programmer" විසින් නිවැරදි කරනු ලැබේ. මෙය "debugging" ලෙස හඳුන්වයි.

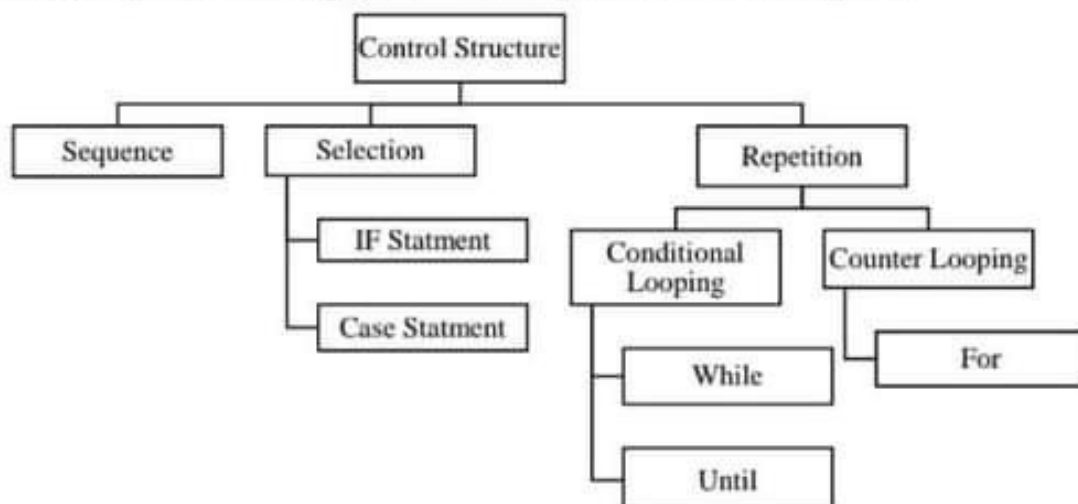
ක්‍රමලේඛ අදාළ දත්ත මත ක්‍රියාත්මක කිරීම

(Execute the program on the input data)

මෙහිදී නිදර්ශකල ක්‍රමලේඛ සත්‍ය ආදාන මත ක්‍රියාකාරී ප්‍රතිදානය ලබාගනු ලැබේ.

පාලන ව්‍යුහය (Control Structure)

පරිගණක ක්‍රමලේඛ තුළ ඇති වගන්ති ක්‍රියාත්මක කරන අනුපිළිවෙල පාලන ව්‍යුහයක් මගින් පාලනය කරනු ලැබේ. ප්‍රධාන පාලන ව්‍යුහ තුනක් ඇත. අනුක්‍රමය, වරණය සහ පුනරාවර්තනය එම පාලන ව්‍යුහ වේ.



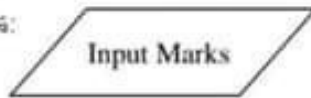
අනුක්‍රමය (Sequence)

ක්‍රමලේඛයක ඇති වගන්ති රේඛානුකූල අනුපිළිවෙලින් දී ඇති ආකාරයට ක්‍රියාත්මක කිරීම අනුක්‍රමය නමින් හැඳින්වේ.

2. ආදානය හෝ ප්‍රතිදානය - Input / Output



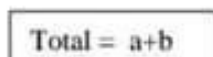
උදාහරණ:



3. සැකසීම - Process



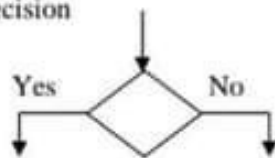
උදාහරණ:



4. දත්ත ගැලීම - Data Flows / Flow line



5. තීරණ - Decision



උදාහරණ:



6. සම්බන්ධකය - Connector



7. අනු සැකසුම - Sub Process



ව්‍යාජ කේත/ කල්පිත කේත (Pseudocode)

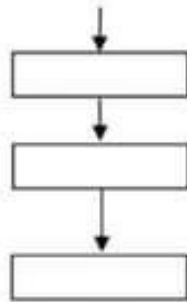
පරිගණක වැඩසටහනක් සකස් කිරීමේදී ක්‍රමලේඛයක් වඩාත් පහසුවෙන් ඇල්ගොරිතමයක් තේරුම් ගැනීම සඳහා ව්‍යාජ කේත භාවිතා කරයි. මෙහිදී ඉංග්‍රීසි භාෂාවේ වචන භාවිතාකර උපදෙස් මාලාව සකස් කෙරේ. මෙම වචන කිසිදු ක්‍රමලේඛ භාෂාවකට භාවිතාකරන වචන නොවූවත් ඒවා බොහෝ දුරට ඒ හා සමාන බවක් පෙන්වුම් කරයි.

ආදානය

Get, read, input

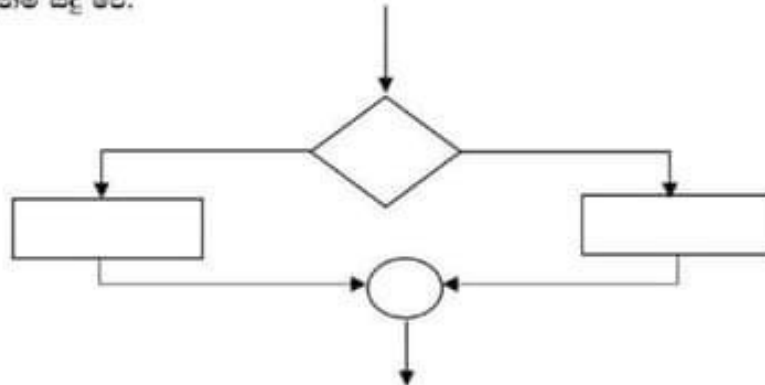
Eg:- Get mark

Read mark



වරණය (Selection)

මෙහිදී තෝරාගැනීම සඳහා විකල්ප ප්‍රකාශන සමූහයක් ඇතිවීම දෙකලද කොන්දේසියක් පරීක්ෂා කර එක් විකල්පයක් පමණක් තෝරාගැනීම සිදු වේ.

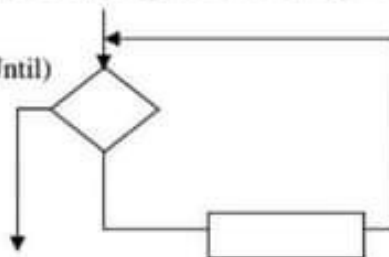


Conditional
(IF – else)
(Switch – Case)

පුනර්තරණය (Repetition / Iteration / Looping)

වගන්ති එකක් හෝ කිහිපයක් නැවත නැවත ක්‍රියාත්මක කිරීම පුනර්තරණය ලෙස හඳුන්වයි.

Loop (While, For, Repeat- Until)



ගැලීම් සටහන් (Flow Chart)

ක්‍රමලේඛයක ක්‍රියාකාරීත්වය අනුපිළිවෙලින් දක්වන රූපමය නිරූපණය ගැලීම් සටහනක් ලෙස හඳුන්වයි. ගැලීම් සටහනක දත්ත ගලායන ආකාරයෙන් ඒවා යම් යම් ක්‍රියාකාරකම් වලට භාජනය වන ආකාරයක් පෙන්වුම් කරයි.

1. ආරම්භ හෝ අවසානය - Start / Stop (end)



උදාහරණ:



ප්‍රතිදානය

Eg:- Output, Display, Print
Display total
Print total

සැකසීම

Eg:- Set, process, calculate
Set total = 0

හස්ත රේඛණය (Hand trace)

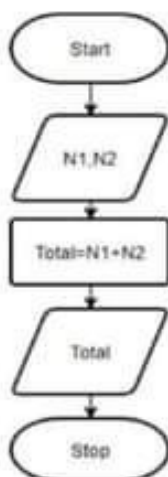
හස්ත ක්‍රම භාවිතයෙන් කේතනය කරන ලද ක්‍රමලේඛනය අදාළ දත්ත ආදානය කිරීමෙන් අදාළ දත්ත භාවිතා කර එය නිවැරදිව ක්‍රියාත්මක කරන ආකාරය බැලීමට නිර්මාණය කරන ලද වගුව හස්ත රේඛණය ලෙස හඳුන්වයි.

Step	Input			Process	Output
	A	B	C	Total= A+B+C	Result
1	10	20	30	10+20+30=60	60
2	70	60	70	70+60+70=200	200
3	60	40	30	60+40+30=130	130
4	10	10	20	10+10+20=40	40

ගැලීම් සටහන් සහ ව්‍යාප්ත කේත සඳහා උදාහරණ

- සංඛ්‍යා දෙකක් ආදානය කර එම සංඛ්‍යා දෙකෙහි එකතුව සෙවීම සඳහා ගැලීම් සටහනක් ඇඳ එයට අදාළ ව්‍යාප්තකයක් සහ පයිතන් ක්‍රමලේඛනයක් ලියන්න.

ගැලීම් සටහන



Begin

Input N1,N2

Process Total=N1+N2

Print Total

End

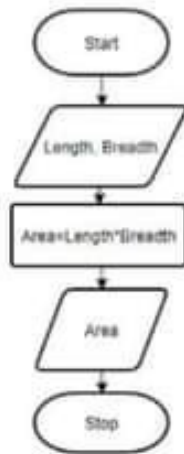
පයිතන් ක්‍රමලේඛනය

```

n1=int(input('Enter number 1:'))
n2=int(input('Enter number 2:'))
total =n1+n2
print('Total is',total)
  
```

- සෘජුකෝණාස්‍රයක වර්ගඵලය සෙවීම සඳහා ගැලීම් සටහනක් ඇඳ එයට අදාළ ව්‍යාප්තකයක් සහ පයිතන් ක්‍රමලේඛනයක් ලියන්න.

ගැලීම් සටහන



ව්‍යාපෘතිය

Begin

Input Length, Breadth

Process $\text{Area} = \text{Length} * \text{Breadth}$

Print Total

End

පයිතන් ක්‍රමලේඛනය

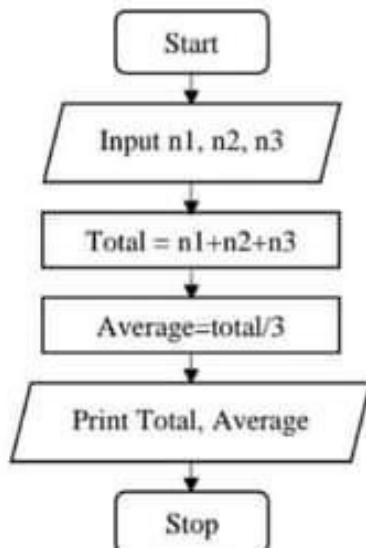
`l=int(input('Enter Length:'))`

`b=int(input('Enter Breadth:'))`

`area =l*b`

`print('Area is', area)`

3. සංඛ්‍යා තුනක් ආදානය කර ඒවායේ එකතුව සහ සාමාන්‍ය සෙවීමට ආදාන ගැලීම් සටහනක් ඇඳ එයට ආදාන ව්‍යාපෘතියක් සහ පයිතන් ක්‍රමලේඛනයක් ලියන්න.



Pseudocode

```

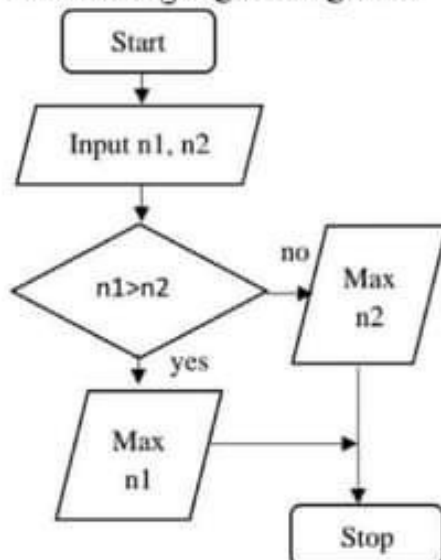
Begin
    Input no1, no2, no3
    Total=no1+no2+no3
    Average=Total/3
    Print Total, Average
End
  
```

Python Code

```

n1=int(input('Enter a number 1: '))
n2=int(input('Enter a number 2: '))
n3=int(input('Enter a number 3: '))
total=n1+n2+n3
average=total/3
print('Total is : ',total)
print('Average is : ',average)
  
```

4. සංඛ්‍යා දෙකක් ආදානය කර විශාලතම සංඛ්‍යාව සෙවීමට ආදාන ගැලීම් සටහනක් ඇඳ එයට ආදාන ව්‍යාපෘතියක් සහ පයිතන් ක්‍රමලේඛනයක් ලියන්න.



Pseudocode

```

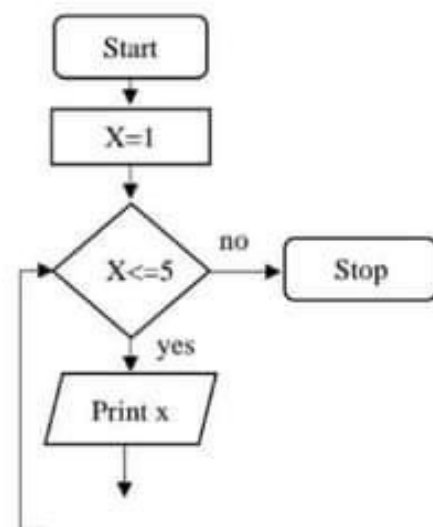
Begin
    Input no1, no2, no3
    If no1>no2 then
        Max=no1
    Else
        Max=no2
    End if
    Print max
End
  
```

Python Code

```

n1=int(input('Enter a number 1: '))
n2=int(input('Enter a number 2: '))
if n1>n2:
    max=n1
else:
    max=n2
print('maximum number is : ',max)
  
```

5. 1 සිට 5 දක්වා සංඛ්‍යාව මුද්‍රනය කිරීමට ආදාන ගැලීම් සටහනක් ඇඳ එයට ආදාන ව්‍යාපෘතියක් සහ පයිතන් ක්‍රමලේඛනයක් ලියන්න.



Pseudocode

```

Begin
    x=1
    while x<=5 do
        print x
        x=x+1
    end while
End
  
```

Python Code

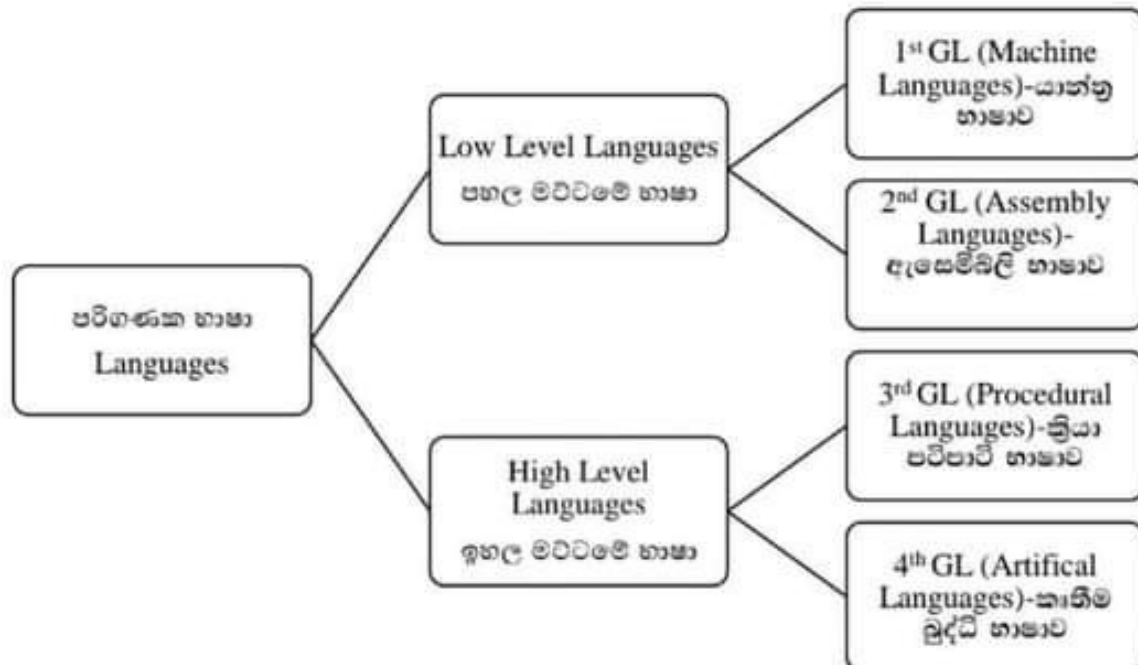
```

x=1
while x<=5:
    print(x)
    x=x+1
  
```

පරිගණක ක්‍රමලේඛන භාෂා

ක්‍රමලේඛනයක් යනු පරිගණකයට යම් කාර්යයක් කිරීම සඳහා ක්‍රමලේඛන භාෂාවක් මගින් ලබාදෙන විද්‍යා මාලාවක එකතුවකි. එනම් ක්‍රමලේඛන භාෂාවක් යනු පරිගණකයට එවන් විද්‍යා ලබාදීම සඳහා සැකසූ කෘතීම භාෂාවකි.

පරිගණක භාෂාවන්ගේ පරිණාමය



පළමු පරම්පරාවේ පරිගණක භාෂාව (Low Level Languages)

- පළමු පරම්පරාවේ පරිගණක භාෂා මගින් ලියන ලද වැඩසටහන් 0 හා 1 යන සංකේත පමණක් භාවිතා කර ලියන ලදී.
- එබැවින් විද්‍යා ඇතුළත් කිරීම ඉතා සංකීර්ණ කාර්යයක් වේ.
- යන්ත්‍ර මත යැපෙන භාෂාවකි.
- ක්‍රමලේඛනයා විසින් පරිගණක දෘඩාංග පිළිබඳ දැනගත යුතු වේ.
- පරිවර්තන මෘදුකාංගයක් අවශ්‍ය නොවේ.
- වැඩසටහන් ක්‍රියාත්මක වීම වේගවත්ය.

දෙවන පරම්පරාවේ පරිගණක භාෂාව

- මෙම පරිගණක භාෂා විද්‍යා යන්ත්‍ර භාෂාවට සාපේක්ෂව සරලය.
- 0 හා 1 මත පදනම් වූ විද්‍යා වෙනුවට සංකේත නාම (mnemonics) භාවිතා වන නිසා මෙය සංකේත භාෂාව ලෙසද හැඳින්වේ.
- Assembler නම් වූ සුවිශේෂී භාෂා පරිවර්තකයක් සංකේතාත්මක භාෂා පරිවර්තනයට යොදාගනී.
- යන්ත්‍රය මත යැපෙන භාෂාවකි.
- මෙම ක්‍රමලේඛන පළමු පරම්පරාවට වඩා ක්‍රියාත්මක වේගය අඩුය.

තෙවන පරම්පරාවේ පරිගණක භාෂාව (High Level Languages)

- මෙම පරිගණක භාෂාව ඉහළ මට්ටමේ පරිගණක භාෂා ලෙස හැඳින්වේ.
- ඉංග්‍රීසි භාෂාවට සමාන පරිගණක භාෂා මෙම ගණයට ගැනේ.
- ක්‍රමලේඛන සම්පාදනය පහසු කරයි.
- යන්ත්‍රය මත යැවීමක් සිදු නොවේ.
- ක්‍රමලේඛනයෙකු පුහුණු කිරීම පහසු වේ.
- පරිවර්තන මෘදුකාංග අවශ්‍ය වේ.

උදා:- 1. සම්පාදක (Compiler)
2. අර්ථවිභාසක (Interpreter)

- FORTAN (Formula Translation)
- COBOL (Common Business Oriental Languages)
- Pascal
- C
- C++

- Java
- Basic (Beginners All Purpose Symbol Instruction Code)
- Python
- Delphi
- Visual Basic

කෘත්‍රීම බුද්ධි සඳහා පරිගණක භාෂා

- නිර්මාණය කළ සඳහා පරිගණක මාධ්‍යය මගින් නිර්මාණය කිරීමට භාවිතා කරයි.
- පරිගණක වලට මිනිසුන් භාවිතා කරන භාෂා තේරුම් ගැනීමට පහසුවෙන් මෘදුකාංග නිර්මාණයට පහසු වේ.
- නිපුණතා පද්ධති නිර්මාණය සඳහා භාවිතා වේ.
- රොබෝ යන්ත්‍රවලට සංවේදී උත්තරයක් ලෙස ප්‍රතිචාර දැක්වීම සඳහා වැඩසටහන් නිර්මාණය කිරීමට භාවිතා කරයි.

පරිගණක භාෂා පරිවර්තන මෘදුකාංග

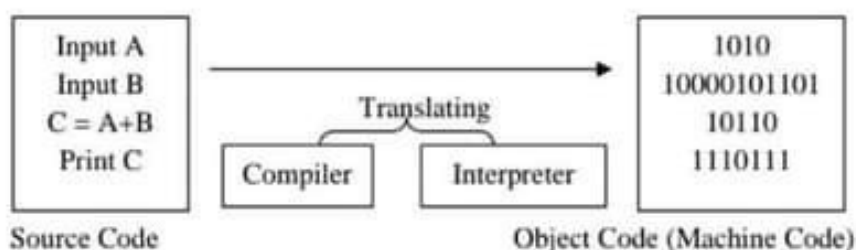
ප්‍රභව කේතය හා විභව කේතය

ප්‍රභව කේතය (Source Program)

පරිගණක භාෂාවකින් ලියන ලද කේතයක් පරිවර්තනය කිරීමට පෙර මුල් පිටපත මෙන් හැඳින්වේ.

විභව කේතය (Object Program)

පරිගණක භාෂාවකින් ලියන ලද කේතයක් යන්ත්‍ර භාෂාව බවට පරිවර්තනය කළ පසු ලැබෙන කේතය විභව කේතය ලෙස හැඳින්වේ.



පරිගණක භාෂාවකින් ලියන ලද කේතයක් යන්ත්‍ර භාෂාව බවට හැරවීමේදී එහි ක්‍රියාකාරීත්වය අනුව ප්‍රධාන වර්ග 3ට බෙදිය හැක. එනම්,

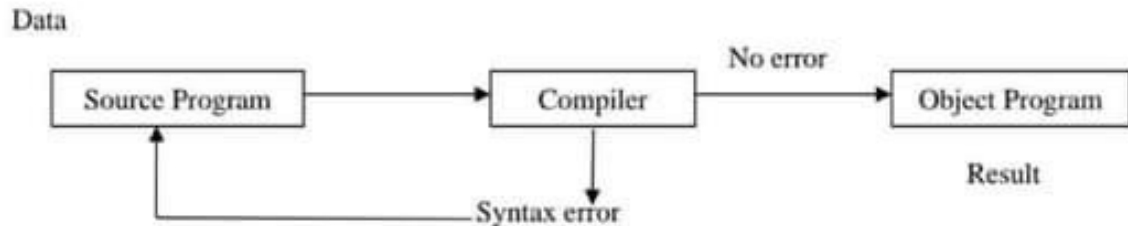
1. කුන්ඩන පරම්පරාවේ පරිවර්තන මෘදුකාංග ලෙස සම්පාදක හා අර්ථ විභාසක හැඳින්විය හැකිය.
2. දෙවන පරම්පරාවේ පරිගණක මෘදුකාංග ලෙස ඇසෙම්බලර් හැඳින්විය හැක.

සම්පාදක (Compiler)

මෙම වර්ගයේ මෘදුකාංග මගින් ක්‍රමලේඛයේ ඇති සියළු විධාන එකවර යන්ත්‍ර භාෂාව බවට හැරවීම Compiler ලෙස හඳුන්වයි. මෙසේ පරිගණකයක වූ පසු එම යාන්ත්‍රික භාෂාවෙන් වූ ක්‍රමලේඛය විධාන දත්ත මත ක්‍රියාත්මක කළහැක. ක්‍රමලේඛය ක්‍රියාත්මක කිරීමේ වේගය අර්ථ වින්‍යාසයට වඩා වැඩිය.

උදා:-

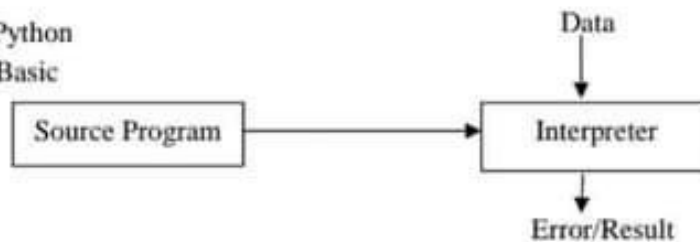
- | | |
|------------|-----------------|
| 1. Pascal | 4. C |
| 2. Fortran | 5. C # |
| 3. C++ | 6. Visual Basic |



අර්ථ වින්‍යාසක (Interpreter)

මෙම පරිවර්තක මෘදුකාංගය මගින් ක්‍රමලේඛයේ අඩංගු වගන්ති පියවරෙන් පියවර යන්ත්‍ර භාෂාව බවට පරිවර්තනය කර විද්‍යාන එසැනින් ක්‍රියාත්මක වීමද සිදු වේ. මෙම ක්‍රමයේදී ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීමට ගන්නා කාලය සම්පාදක මගින් පරිවර්තනය කළ ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීමට ගන්නා කාලය වැඩිය.

- උදා:-
1. Python
 2. Basic

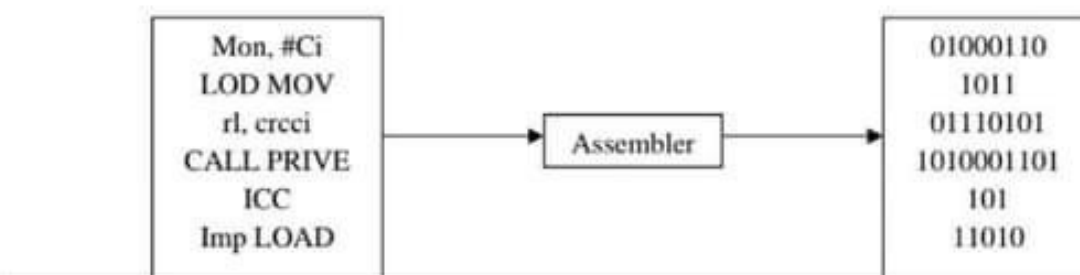


සම්පාදක හා අර්ථ වින්‍යාසක අතර වෙනස්කම්

සම්පාදක (Compiler)	අර්ථ වින්‍යාසක (Interpreter)
ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීමට පෙර සම්පූර්ණ ක්‍රමලේඛයක් එකවර යන්ත්‍ර භාෂාව බවට හැරවිය යුතුය.	ක්‍රමලේඛයක් පියවරෙන් පියවර යන්ත්‍ර භාෂාවක් බවට හරවා ගත අතර එසැනින්ම ක්‍රියාත්මක වීමද සිදුවේ.
ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීමේ වේගය වැඩිය.	ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීමේ වේගය අඩුය.

එසෙම්බ්ලි (Assembler)

ප්‍රාථමික මට්ටමේ පරිගණක භාෂාවකින් ලියන ලද ක්‍රමලේඛයක් (mnemonics) යන්ත්‍ර භාෂාව බවට පරිවර්තනය කිරීම එසෙම්බ්ලි ලෙස හඳුන්වයි.



ක්‍රමලේඛන දෝෂ - Programming Errors (Bugs)

1. කාරක දෝෂ (Syntax Error) (Compile-time error)
 1. වැරදි මූලපද එක් කිරීම (Wrong Keyword)
 2. වැරදි විරාම ලකුණු භාවිතා කිරීම (Wrong Punctuation Marks)
2. ක්‍රමලේඛයක් ක්‍රියාත්මක වීමේදී ඇතිවන දෝෂ (Run-time error)
 1. සංඛ්‍යාවක් බිංදුවෙන් බෙදීම (Division by Zero)
 2. සෘණ සංඛ්‍යා වල වර්ග මූලය සෙවීම (Find Square root in negative number)
3. තාර්කික දෝෂ (Logical error)
 1. Unexpected Output (අපේක්ෂිත ප්‍රතිඵල නොපෙන්වීමේදී ඇතිවන දෝෂ)

පයිතන් ක්‍රමලේඛන භාෂාව (Python Programming Language)

මෙය ඉහළ මට්ටමේ ක්‍රමලේඛ භාෂාවක් වන අතර මෙම භාෂාව 90 දශකයේ මුල් භාගයේ නෙදර්ලන්තයේ CWICN (Research Institute of Mathematics and Computer Science) ආයතනයේ **Guido Van Rossum** විසින් නිර්මාණය කරන ලදී. මෙම භාෂාව අර්ථ වින්‍යාසය මත පදනම් වූ වස්තුගත (Object Oriented) භාෂාවක් වන අතර පරිපාටිගත භාෂාවක් ලෙසද යොදාගත හැක. විවෘත මෘදුකාංග මූලාශ්‍ර ක්‍රමලේඛන භාෂාවක් වන අතර <http://www.python.org> වෙබ් අඩවියෙන් නොමිලේ බාගතකර ගත හැක. පයිතන් ක්‍රමලේඛන භාෂාව පහත මෙහෙයුම් පද්ධති මත ක්‍රියාත්මක කළ හැක.

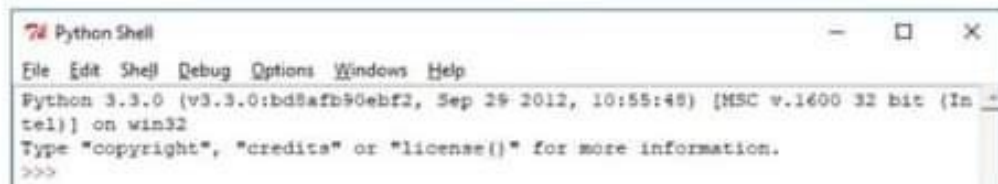
- | | |
|-----------------|------------------------|
| 1. Linux / Unix | 4. Amiga |
| 2. Mac OS X | 5. Palm Handhelds |
| 3. OS / 2 | 6. Nokia mobile phones |

Python Shell විවෘත කිරීම

Start→ All Program→ python 3.3→ IDLE (python GUI)

Python හි අතුරු මුහුණත් ආකාර 2කි.

1. රූපක අතුරු මුහුණත
2. විධාන රේඛා (Command line) අතුරු මුහුණත

**Copyright විධානය**

74 Python Shell

File Edit Shell Debug Options Windows Help

Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32

Type "copyright", "credits" or "license()" for more information.

```
>>> copyright
```

Copyright (c) 2001-2012 Python Software Foundation.
All Rights Reserved.

Copyright (c) 2000 BeOpen.com.
All Rights Reserved.

Copyright (c) 1995-2001 Corporation for National Research Initiatives.
All Rights Reserved.

Copyright (c) 1991-1995 Stichting Mathematisch Centrum, Amsterdam.
All Rights Reserved.

```
>>>
```

පයිතන් shell එකෙන් ඉවත් වීම

මේ සඳහා quit() හෝ exit() විධානය භාවිතා කරයි.

74 Python Shell

File Edit Shell Debug Options Windows Help

Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32

Type "copyright", "credits" or "license()" for more information.

```
>>> exit()
```

Hello Python මෙය මුද්‍රණය කිරීම

74 Python Shell

File Edit Shell Debug Options Windows Help

Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32

Type "copyright", "credits" or "license()" for more information.

```
>>> print('Hello python')
```

Hello python

```
>>> |
```

74 Python Shell

File Edit Shell Debug Options Windows Help

Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32

Type "copyright", "credits" or "license()" for more information.

```
>>> print 'Hello python'
```

SyntaxError: invalid syntax

```
>>>
```

Python නව සංස්කරණ වලදී print විධානය පසුව වරහනක් () තුළ මුද්‍රණය කළ යුතු වගන්තිය ලිවිය යුතුය. නැතහොත් ඉහත ආකාරයේ දෝෂ පණිවිඩයක් නිකුත් කරයි. මුද්‍රණය කළ යුත්තේ වගන්තියක් (String Values) නම් එය උද්දාත තුළ දැක්විය යුතුය. පයිතන් භාෂාවේදී තනි උද්දාතය ('Hello python') , ද්විත්ව උද්දාතය ("Hello python") සහ ත්‍රිත්ව උද්දාතය ('''Hello python''') වලට මෙය.

Python Shell තුළ ගණිත කර්ම සිදුකළ හැක.



```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 5+2
7
>>> 5-2
3
>>> 5*3
15
>>> x=5
>>> y=3
>>> x+y
8
>>> 5/2
2.5
>>> 5%2
1
>>> 5**2
25
>>> 5//2
2
>>> print(x+y)
8
>>>
```

ක්‍රමලේඛයක් ලිවීම සඳහා නව ගොනුවක් විවෘත කිරීම

File → New Window (Ctrl+N)

Print ("Hello World")

ක්‍රමලේඛයක් ගබඩා කිරීම

File → Save → (extension → .py) .py හෝ .pyw ගොනු දිගු භාවිතා කළ හැක.

ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීම

Run → Run Module (F5)

>>>Hello World

සබැඳි නම සහ ලිපිනය මුද්‍රණය කිරීම සඳහා පහත ක්‍රමලේඛනයක් ලිවීම.

```
e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
print('Aruna Herath')
print('ICT Institute')
print('Fair Junction')
print('Polonnaruwa')

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Aruna Herath
ICT Institute
Fair Junction
Polonnaruwa
>>> |
```

දත්තයක් යතුරු පුවරුව මගින් ක්‍රමලේඛයට ආදානය කිරීම සඳහා **input** විද්‍යාය භාවිතා කරයි.

```
e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
fname=input('Enter your first name: ')
lname=input('Enter your last name: ')
fullname=fname+' '+lname
print('Your name is',fullname)

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Enter your first name: Aruna
Enter your last name: Herath
Your name is Aruna Herath
>>> |
```

නම සහ වයස ඇතුළත් කිරීම සඳහා පහත ක්‍රමලේඛනයක් ලිවීම

```
name=input('Enter your first name: ')
age=input('Enter your age: ')
print('Your name is',name, 'your age is',age)

>>>
Enter your first name: Aruna
Enter your age: 34
Your name is Aruna your age is 34
>>>
```

සංඛ්‍යා 2ක් ආදානය කර ඒවායේ එකතුව සෙවීම

```
74 e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
no1=int(input('Enter number 1: '))
no2=int(input('Enter number 2: '))
total=no1+no2
print('Total is',total)
Ln: 4 Col: 23
```

විෂයන් තුනක ලකුණු ඇතුළත් කර ඒවායේ එකතුව සහ සාමාන්‍ය සෙවීම

```
74 e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
m1=int(input('Enter Accounts marks: '))
m2=int(input('Enter Econ Marks: '))
m3=int(input('Enter Ict Marks: '))
total=m1+m2+m3
average=total/3
print('Total is',total)
print('Average is',round(average,2))
```

සෘජුකෝණාස්‍රයක වර්ගඵලය සහ පරිමිතිය සෙවීම

```
74 e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
b=int(input('Enter Breadth: '))
l=int(input('Enter Length: '))
area=b*l
perimeter=2*(b+l)
print('Area is',area)
print('perimeter is',perimeter)
```

වෘත්තයක වර්ගඵලය සහ පරිමිතිය සෙවීම

```
74 e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
r=int(input('Enter radius: '))
pi=3.14 #pi=22/7
area=pi*r**2
perimeter=2*pi*r
print('Area is',area)
print('perimeter is',perimeter)
Ln: 5 Col: 21
```

කාරක රීති සහ ගබ්දාර්ථ (Syntax and Semantics)

පරිගණක භාෂා නිර්වචනය කිරීම සඳහා ප්‍රධාන සංකල්ප 2ක් අවශ්‍ය වේ.

1. කාරක රීති (Syntax)
2. ගබ්දාර්ථ (Semantics)

පරිසීමක (Delimiters)

ප්‍රකාශන කොටස් වෙන්කර දැක්වීම සඳහා පරිසීමක භාවිතා කරයි. පයිතන් භාෂාවේ භාවිතා වන පරිසීමක කිහිපයක් පහත දැක්වේ.

()	*	%=
[]	=	<=
{ }	!	^=
@	+=	>>=
,	-=	<<=
:	/=	**=
.	//=	!=

උදා:-
`a=5`
`b=10`
`a=a+b` $\rightarrow a+ = b$

`print ("Answer is" , a)`
 Statement Variable
 Delimiters

බහු පේළි ප්‍රකාශන (Multi – Line Statements)

Explicit Line Join (\)

ක්‍රමලේඛනයක කිසියම් පේළියක් ' \ ' යන අනු ලක්ෂණයෙන් අවසාන වී ඇති අතර එම පේළිය ඊට පසුව ඇති පේළිය සමඟ සම්බන්ධ කරනු ලබයි.

e.g :- `print('ict \`
`institute')`

Impact Line Join – () [] { }

- [], () හා { } යන වරහන් තුළ වූ ප්‍රකාශන පේළි කීපයක් කේත කළ හැක.

e.g :- `x=[5,2,`
`3,4]`

පයිතන් හි භාවිතා කළ හැකි උද්දාන (Quotes)

Python භාෂාවේ භාවිතා කරන අක්ෂර පාඨ හෝ පේද දැක්වීම සඳහා ' , " , "" "" භාවිතා කරයි.

උදා:-

- `a = ' It is a cat'`
- `a = " It is a Cat "`
- `a = '''It's a Cat'''` $\rightarrow$ `a = """"It's a Cat""""` (තනි උද්දාන තුනක් හෝ ද්විත්ව උද්දාන තුනක් භාවිතා කළ හැක.)

පේළි සහ අනුපේද (Lines and Indentation)

පයිතන් භාෂාවේ භාවිතා වන ප්‍රකාශන අනුක්‍රමයක් කට්ටියක් ලෙස හැඳින්වේ. ලේඛනයක ඇති මෙවැනි කොටසක් ලිවීමේදී අනුපේදනය භාවිතා කිරීම ඉතා වැදගත් වේ. මෙහිදී ක්‍රමලේඛනයක් ඉදිරියේ ඇති අවසානය මගින් අනුපේදයක් දක්වයි.

e.g:-
`If True:`
`print ("True")`
`Else:`
`print ("False")`

පයිතන් හි භාවිතා වන විවරණ (Comments)

ක්‍රමලේඛනයේ පිළිබඳ විස්තර ක්‍රමලේඛනයේ තුළට ලේඛන ගත කිරීම සඳහා විවරණ භාවිතා කරයි. විවරණ ක්‍රමලේඛනයක ඇති අනෙකුත් ප්‍රකාශන මෙන් ක්‍රියාත්මක නොවේ. ක්‍රමලේඛනයේ තුළ ඇති විවරණ අන්තර්ගත කිරීමේ මූලික පරමාර්ථය වන්නේ මෙම ක්‍රමලේඛය පරීක්ෂණය කරන්නෙකුට එය පහසුවෙන් තේරුම් ගැනීමට අවශ්‍ය විස්තර සපයා දීමටය.

උදා:-
`# First Comment`
`print ("Hello, python") # Second Comment`

කාරක රීති (Syntax)

ක්‍රමලේඛ භාෂාවක් කාරක රීති මගින් ක්‍රමලේඛ භාෂාවේ ඇති ව්‍යාකරණ වලට අනුව සංකේත සම්බන්ධ කළයුතු ආකාරය නිර්වචනය කරයි. මෙම කාරක රීතිවලින් නිර්වචනය වන්නේ සංකේතවල ව්‍යුහයන්ගේ නිර්වචනයාව පිළිබඳ පමණක් වන අතර ඒවායේ අර්ථය පිළිබඳ සැලකිල්ලක් නොදක්වයි.

උදා:- Dog is a bird

ශබ්දාර්ථ (Semantics)

මෙමගින් භාෂාවක ඇති ව්‍යාකරණ ව්‍යුහයන්ට අනන්‍ය අර්ථ පවරනු ලැබේ. එමගින් භාෂාවකින් සම්පාදනය කළ වැඩසටහන් ක්‍රියාත්මක කිරීමේදී අනුගමනය කළයුතු වර්ගව ශබ්දාර්ථ මගින් නිර්වචනය කෙරේ.

පයිතන් හඳුන්වන (Python Identifiers)

හඳුන්වනයක් යනු විචල්‍යයක්, නියතයක්, ක්ලාස්, මොඩියුල හෝ ශ්‍රිතයක් (Variable, Constant, Function, Class, Module) හැඳින්වීම සඳහා යොදාගන්නා නාමයකි. Identifier නිර්මාණය කිරීමේදී අනුගමනය කළයුතු නීති පහතින් දක්වේ.

- හඳුන්වනයක් ඉංග්‍රීසි භාෂාවෙහි අක්ෂරයකින් හෝ යටි අර underscore (_) මගින් දක්විය යුතුය.
- පළමු අක්ෂරයට පසු වන අක්ෂරය සඳහා ඉහත ආකාරයේ අක්ෂර හා 0-9 වූ ඉලක්කම්ද භාවිතා කළ හැකිය.
- නිශ්චය හැකි අනුලක්ෂණ සංඛ්‍යාවේ පිමාවක් නොමැත.
- Python හි භාවිතා වන මූල පද භාවිතා කළ නොහැක.
- ඉංග්‍රීසි භාෂාවේ Simple අකුරු හා Capital අකුරු එකිනෙකට වෙනස් අක්ෂර ලෙස සලකනු ලැබේ.(Mark ≠ mark)
- @, \$, % යන අනුලක්ෂණ භාවිතා කළ නොහැක.

Python මූල පද/අවුරුතු පද (Reserved Words)

Python භාෂාවේද අනෙක් බොහෝ පරිගණක භාෂාවල මෙන් මූලපද ලෙස අර්ථ දක්වූ වචන ලැයිස්තුවක් වේ. ක්‍රමලේඛන තොඩනැගීමේදී එම වචනවල භාවිතය ඒවායේ අනුරූප අර්ථයන්ට සිදුකළ යුතුය. python භාෂාවේ ඇති මූලපද ලැයිස්තුවක් පහත දක්වේ.

and	not
as	or
assert	pass
break	print
class	raise
continue	return
def	try
del	while
elif	with
else	yield
except	true
exec	false
finally	None
for	
from	
global	
if	
import	
in	
is	
lambda	

පයිතන් හි විචල්‍ය සහ දත්ත වර්ග Python Variables and Data Types

ක්‍රමලේඛයක් ක්‍රියාත්මක කිරීමේදී දත්තයක් තාවකාලිකව ප්‍රධාන මතකයේ තබාගැනීමට අවශ්‍ය වේ. මෙසේ දත්තයක් මතකයේ තාවකාලිකව තබාගැනීම සඳහා විචල්‍ය භාවිතා වේ. පරිගණකයේ ප්‍රධාන මතකයේ නිශ්චිත කොටසකට ප්‍රවේශවීම සඳහා උපයෝගී කරගත හැකි සංකේත නාමයක් ලෙස විචල්‍ය දැක්විය හැකිය. මෙලෙස විචල්‍යක් මතකයේ නිශ්චිත කොටසකට තාවකාලිකව අනුබද්ධ වීම විචල්‍ය උපයෝගී කරගෙන එම අදාළ මතක කොටසේ දත්ත තාවකාලිකව ගබඩා කිරීම හා නැවත ලබා ගැනීම සිදුකළ හැකිය.

විචල්‍යකට අගයක් ආදේශ කිරීම

විචල්‍යකට අගයක් ආදේශ කිරීමේදී සමාන ලක්ෂණ භාවිතා කරයි. එය Assign Operators ලෙස හඳුන්වයි.

Variable Name = Value

උදා:-

1. mark = 98
2. age = 18
3. pi = 3.14
4. name = Python
5. $\left. \begin{array}{l} x = 5 \\ y = \text{'ICT'} \\ z = 72 \end{array} \right\} \begin{array}{l} x = 1 \\ y = 1 \\ z = 1 \end{array} \right\} x=y=z=1$
 $\left. \begin{array}{l} a = 5 \\ b = 6 \end{array} \right\} a,b=5,6$
8. $\left. \begin{array}{l} x = 1.5 \\ y = \text{'String'} \\ z = 1 \end{array} \right\} x,y,z = 1.5, \text{'String'}, 1$

පයිතන් විචල්‍ය වර්ග (Python Variable Types)

කලින් නිර්වචනය කළ ලක්ෂණ හා මෙහෙයවන (Operators) සහිත දත්ත සමූහයක් දත්ත ප්‍රථමයක් ලෙස හැඳින්විය හැකිය. දත්ත භාවිතයට පෙර ඒවායේ ප්‍රථම දැනගත යුතුය.

පයිතන් වල භාවිතා වන දත්ත වර්ග (Python standard data types)

→ Numbers

- Integral
 - Integer -int
 - Boolean -bool
- Real -float
- Complex -complex

→ Sequences

- Immutable Sequences
 - String -str
 - Tuple -tuple
 - Byte -byte
- Mutable Sequences
 - Lists - list
 - Byte Arrays - bytearray

- Set types
 - Sets -set
 - Frozen Set -frozenset
- Mappings
 - Dictionaries -dict

Mutable

Python හි වස්තු ස්වරූපයකට අයත් වස්තූන්ගේ අගයන් එම වස්තු නිර්මාණය කිරීමෙන් පසු ඕනෑම අවස්ථාවක වෙනස් කළ හැකි නම් Mutable සංයෝග අයත් වේ.

උදා:-

1. Dictionaries
2. Set
3. Lists

```
e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
x=[5,6,'ict','aruna']
x[1]=8
x[3]='Herath'
print(x)
```

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (In
tel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
[5, 8, 'ict', 'Herath']
>>> |
```

Immutable

වස්තුවක් නිර්මාණය කිරීමෙන් පසු ඒවායේ අගයන් වෙනස් කළ නොහැකි නම් එම දත්ත Immutable සංයෝග අයත් වේ.

- Number
- String
- Tuple
- Frozen test

```
e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
x=(5,6,'ict','aruna')
x[1]=8
x[3]='Herath'
print(x)
```

- Set types
 - Sets -set
 - Frozen Set -frozenset
- Mappings
 - Dictionaries -dict

Mutable

Python හි වස්තු ස්වරූපයකට අයත් වස්තූන්ගේ අගයන් එම වස්තු නිර්මාණය කිරීමෙන් පසු ඕනෑම අවස්ථාවක වෙනස් කළ හැකි නම් Mutable සංයෝග අයත් වේ.

උදා:-

1. Dictionaries
2. Set
3. Lists

```
e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
x=[5,6,'ict','aruna']
x[1]=8
x[3]='Herath'
print(x)
```

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (In
tel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
[5, 8, 'ict', 'Herath']
>>> |
```

Immutable

වස්තුවක් නිර්මාණය කිරීමෙන් පසු ඒවායේ අගයන් වෙනස් කළ නොහැකි නම් එම දත්ත Immutable සංයෝග අයත් වේ.

- Number
- String
- Tuple
- Frozen test

```
e3.py - C:/Python33/e3.py
File Edit Format Run Options Windows Help
x=(5,6,'ict','aruna')
x[1]=8
x[3]='Herath'
print(x)
```

```

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd9a9b90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Traceback (most recent call last):
  File "C:/Python33/e3.py", line 2, in <module>
    x[1]=3
TypeError: 'tuple' object does not support item assignment
>>>

```

Iterable

එක් වරකට එක් අවයවය බැගින් මුදාහැරිය හැකි දත්ත ඵකතුවක් iterable ලෙස හැඳින්වේ.

- List
- String
- Tuple

සංඛ්‍යා සහිත දත්ත (Python Number)

සංඛ්‍යාත්මක අගයන් ගබඩා කිරීම සඳහා භාවිතා කරයි. මෙය මගින් තබාගත හැකි කුඩාම සහ විශාලතම සංඛ්‍යාව පරිගණක සංඛ්‍යා නිරූපනය සඳහා ඇති ශීමාවන් මත රඳා පවතී.

- Int - ධන හෝ ඍණ පූර්ණ සංඛ්‍යා සඳහා භාවිතා කරයි. උදා:- 1,2, -5
- Long - විශාල පූර්ණ සංඛ්‍යා නිරූපනය සඳහා භාවිතා කරයි. සංඛ්‍යාවේ අවසානයට L හෝ l භාවිතා කරයි. උදා:- 51924361L
- Float - දශම සංඛ්‍යා (තාත්වික සංඛ්‍යා) දැක්වීම සඳහා භාවිතා කරයි. උදා:- 2.25, 5.0, -2.3, 2.5e2 = 2.5 x 10² = 250
- Complex - සංකීර්ණ සංඛ්‍යා දැක්වීම සඳහා භාවිතා කරයි. සංඛ්‍යාවේ අවසානයට J හෝ j යොදයි. උදා:- 3.14j

```

76 *ex4.py - C:/Python33/ex4.py
File Edit Format Run Options Windows Help
x = 1      # int
y = 2.8    # float
z = 1j     # complex
print(type(x))
print(type(y))
print(type(z))
Ln: 6 Col: 14

```

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
<class 'int'>
<class 'float'>
<class 'complex'>
>>> |
```

Ln: 8 Col: 4

```
ex4.py - C:/Python33/ex4.py
File Edit Format Run Options Windows Help
x = 1
y = 35656222554887711
z = -3255522

print(type(x))
print(type(y))
print(type(z))
```

Ln: 6 Col: 14

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
<class 'int'>
<class 'int'>
<class 'int'>
>>>
```

Ln: 7 Col: 13

```
ex4.py - C:/Python33/ex4.py
File Edit Format Run Options Windows Help
x = 1.10
y = 1.0
z = -35.59

print(type(x))
print(type(y))
print(type(z))
```

Ln: 7 Col: 14

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
<class 'float'>
<class 'float'>
<class 'float'>
>>> |
```

Ln: 8 Col: 4

```
ex4.py - C:/Python33/ex4.py
File Edit Format Run Options Windows Help
x = 35e3
y = 12E4
z = -87.7e100

print(type(x))
print(type(y))
print(type(z)) |
```

Ln: 7 Col: 14

```
Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
<class 'float'>
<class 'float'>
<class 'float'>
>>> |
```

Ln: 8 Col: 4

```
ex4.py - C:/Python33/ex4.py
File Edit Format Run Options Windows Help
x = 3+5j
y = 5j
z = -5j

print(type(x))
print(type(y))
print(type(z)) |
```

Ln: 7 Col: 14


```

Python Shell
File Edit Shell Debug Options Windows Help
Python 3.3.0 (v3.3.0:bd8afb90ebf2, Sep 29 2012, 10:55:48) [MSC v.1600 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
<class 'complex'>
<class 'complex'>
<class 'complex'>
>>>
Ln: 8 Col: 4

```

Python Number Log10()

```
import math
print(math.log10(100))
```

Output

2.0

Python Number min()

```
x=10,2,25,4
print(min(x))
```

Output

2

Python Number max()

```
x=10,2,25,4
print(max(x))
```

Output

25

```
import math
print(math.pow(5,2))
```

Output

25.0

Python Number round()

```
x=25.25678
print(round(x,2))
```

Output

25.27

Python Number sqrt()

```
import math
print(math.sqrt(25))
```

Output

5.0

Python Number pow()

අක්ෂර සහිත දත්ත (Python String)

උද්දාහරණයක් ලෙස දක්වනු ලබන අනුලක්ෂණයක් හෝ අනුලක්ෂණ කාණ්ඩයක් මෙයට අයත් වේ.

උදා:- '5', 'aruna', 'I', 'ict institute'

```
>>> '5'
'5'
>>> 'aruna'
'aruna'
>>> 'ict institute'
'ict institute'
>>> x='ict'
>>> print(x)
ict
>>> print(x, 'institute')
ict institute
>>> print(x+'institute')
ictinstitute
>>> print(x+' '+institute')
ict institute
>>> print(x*2)
ictict
>>> print(len(x))
3
>>> print(type(x))
<class 'str'>
>>> print(x[0])
i
```

```
>>> x='Python is an interpreted'
>>> print("%s is a programming language"%x[:6])
Python is a programming language
>>> x[:6]
'Python'
>>> max(x)
'y'
>>> min(x)
' '
>>> min('Ict')
'I'
>>> x.count('i')
2
>>> x.count('e')
3
>>> x.find('h')
3
>>> x[::-1].startswith('d')
True
>>>
```

```
>>> print('%s is %d years old.'%(nimal,16))
nimal is 16 years old.
>>> print('ict'+\
'institute')
ictinstitute
>>> print('ict\ninstitute')
ict
institute
>>> print('ict\tinstitute')
ict      institute
```

```
>>> x='ict institute'
>>> y='polonnaruwa'
>>> print(x,y)
ict institute polonnaruwa
>>> print(x[-1])
e
>>> print([:-1])
SyntaxError: invalid syntax
>>> print(x[:-1])
ict institut
>>> print(x[::])
ict institute
>>> print(x[::2])
itisue
>>> print(x[::3])
i ste
>>> print(x[-1:])
e
>>> "ict"+"institute"
'ictinstitute'
>>> 'ict institute'[2:]
't institute'
>>> print([2])
SyntaxError: invalid syntax
>>> print(x[2:])
t institute
>>>
```

```
>>> x='ict institute'
>>> print(x.title())
Ict Institute
>>> print(x.upper())
ICT INSTITUTE
>>> print(x.lower())
ict institute
>>> print(x.capitalize())
Ict institute
>>> print(x.replace('ict','abc'))
abc institute
>>> print(x.split(' '))
['ict', 'institute']
```

end() විධානය

a='ict'

b='institute'

c='Polonnaruwa'

print(a ,end=',')

print(b ,end=',')

print(c ,end=',')

ප්‍රතිඵලය

ict, institute, polonnaruwa,

```
>>> x='ict'
>>> for i in x:
    print(i)
```

```
i
c
t
>>> print(x.split(','))
['ict', 'bit', 'fit', 'cit']
>>> y=x.split(',')
>>> for i in y:
    print(i)
```

```
ict
bit
fit
cit
>>> |
```

දත්ත ව්‍යුහ (Data Structure)

ක්‍රමලේඛනයක් තුළ දත්ත කාර්යක්ෂමව යොදා ගැනීම සඳහාත් දත්ත පිළිවෙලකට සංවිධානය කිරීම සඳහාත් දත්ත ව්‍යුහ භාවිතා කරයි. පයිතන් ක්‍රමලේඛන භාෂාවේදී භාවිතා කරනු ලබන දත්ත ව්‍යුහ ආකාර කිහිපයකි.

1. List
2. Tuple
3. Dictionary
4. Set

List

පයිතන් ක්‍රමලේඛන භාෂාවේ භාවිතා වන ප්‍රධාන දත්ත ව්‍යුහය මෙයයි. List දැක්වීම සඳහා කොටු වරහන [] භාවිතා කරයි. list එකක් තුළ ඇති සෑම අංගයක් සඳහාම සුවි අගයක් (index) ඇත. පළමු සුවි 0න් ආරම්භ වේ. පයිතන් List එකක් තුළ එකිනෙකට වෙනස් දත්ත ප්‍රථමවලට අදාළ දත්ත ඇතුළත් කළ හැකිය. List එකක් තුළ ඇති අයිතම කොමාවක් මගින් වෙන්කරනු ලබයි.

```
>>> list1=['ict','aruna','polonnaruwa']
>>> list2=['a','b','c','d','e','f']
>>> list3=[5,6,10,6,2.25]
>>> print(list1)
['ict', 'aruna', 'polonnaruwa']
>>> print(list2)
['a', 'b', 'c', 'd', 'e', 'f']
>>> print(list3)
[5, 6, 10, 6, 2.25]
>>> print(list1,list2,list3)
['ict', 'aruna', 'polonnaruwa'] ['a', 'b', 'c', 'd', 'e', 'f'] [5, 6, 10, 6, 2.25]
```

```
>>> print(list1+list2)
['ict', 'aruna', 'polonnaruwa', 'a', 'b', 'c', 'd', 'e', 'f']
>>> print(list1*2)
['ict', 'aruna', 'polonnaruwa', 'ict', 'aruna', 'polonnaruwa']
>>> print(list1+['a','r','u','n','a'])
['ict', 'aruna', 'polonnaruwa', 'a', 'r', 'u', 'n', 'a']
>>> for i in list1:
    print(i)
```

```
ict
aruna
polonnaruwa
>>> for i in list1:
    print(i,end="")
```

```
ictarunapolonnaruwa
>>> x=['aruna','ict',5,6]
>>> print(x[0])
aruna
>>> print(x[1])
ict
>>> print(x[-1])
6
>>> print(x[:2])
['aruna', 'ict']
>>> print(x[:])
['aruna', 'ict', 5, 6]
>>> print(x[1:3])
['ict', 5]
```

List සඳහා භාවිතා කළ හැකි ශ්‍රිත

```
>>> #maximum item of a list
>>> x=[5,6,10,2,3]
>>> y=['a','b','c','abc','A']
>>> print(max(x))
10
>>> print(max(y))
c
>>> #minimum item of a list
>>> print(min(x))
2
>>> print(min(y))
A
```


append()

සකස් කර ඇති List එකක අවසානයට නව අයිතමයක් එක් කිරීම සඳහා append භාවිතා කරයි.

```
>>> animals=['cat','dog']
>>> animals.append('cow')
>>> print(animals)
['cat', 'dog', 'cow']
>>> animals.append('fish')
>>> print(animals)
['cat', 'dog', 'cow', 'fish']
```

ගැටළු :

1. සිසුන් පස් දෙනෙකුගේ නම් සහ ලකුණු ඇතුළත් කිරීම සඳහා ලැයිස්තු දෙකක් සකස් කර අවසානයේ සිසුන්ගේ නම් සහ එක් එක් සිසුවා ලබාගත් ලකුණු දැක්වීමටත් එම ලකුණු වල එකතුව සහ සාමාන්‍යය දැක්වීමටත් පයිතන් ක්‍රමලේඛනයක් ලියන්න.

```
name=[]
marks=[]
total=0
average=0
c=1
while c<=5:
    n=str(input('Enter name: '))
    name.append(n)
    m=int(input('Enter marks: '))
    marks.append(m)
    c=c+1
x=0
print('name \t\t marks')
print('-----')
while x<5:
    print(name[x],'\t\t',marks[x])
    total=total+marks[x]
    x=x+1
average=total/5
print('-----')
print('Total marks is : ',total)
print('Average marks is : ',average)
```

Enter name: Amal
Enter marks: 90
Enter name: Kamal
Enter marks: 95
Enter name: Saman
Enter marks: 85
Enter name: Lalith
Enter marks: 94
Enter name: Bimal
Enter marks: 48

name	marks
Amal	90
Kamal	95
Saman	85
Lalith	94
Bimal	48

Total marks is : 412
Average marks is : 82.4

2. පරිගණක වෙළඳ සැලකිත් හාණ්ඩ 5ක් ලබාගෙන ඒවායින් ලබාගත් ඒකක ගනන සහ ඒකකයක මිලද ඇතුළත්කර හාණ්ඩ සියල්ල සඳහා ගෙවිය යුතු මුදල ගණනය කිරීම සඳහා පයිතන් ක්‍රමලේඛනයක් ලියන්න. හාණ්ඩවල නම් සහ ඒකකයක මිල ලැයිස්තු දෙකකට ඇතුළත් විය යුතුය. ප්‍රතිදානය බිල්පතක් ආකාරයෙන් විය යුතුය.


```

#products
p=[]
up=[]
u=[]
c=1
total=0
while c<=5:
    product=str(input('Enter product name: '))
    p.append(product)
    price=float(input('Enter price of product: '))
    up.append(price)
    unit=int(input('Enter number of units: '))
    u.append(unit)
    c+=1
#find the amount
print()
print("-----")
print("ICT Computers (Pvt) Ltd")
print("Hospital Junction - Polonnaruwa")
import datetime
now=datetime.datetime.now()
print(now.strftime('%Y-%m-%d %H:%M:%S'))
print("-----")
print("Item Name\tQuantity\tUnit Price\tAmount")
print("-----")
for i in range (5):
    print(p[i],'\t', u[i], '\t',up[i],'\t',u[i]*up[i])
    total=total+u[i]*up[i]
print("-----")
print('Total price: ',total)

```

```

Enter product name: CD
Enter price of product: 15
Enter number of units: 100
Enter product name: DVD
Enter price of product: 25
Enter number of units: 100
Enter product name: Ram
Enter price of product: 750
Enter number of units: 2
Enter product name: HDD
Enter price of product: 900
Enter number of units: 3
Enter product name: VGA
Enter price of product: 500
Enter number of units: 4

```

```

-----
ICT Computers (Pvt) Ltd
Hospital Junction - Polonnaruwa
2021-05-22 10:15:10
-----

```

Item Name	Quantity	Unit Price	Amount
CD	100	15.0	1500.0
DVD	100	25.0	2500.0
Ram	2	750.0	1500.0
HDD	3	900.0	2700.0
VGA	4	500.0	2000.0

```

-----
Total price: 10200.0

```

count()

List එකක ඇති අවයවයන් කොපමණ වාර ගණනක් යෙදී තිබේ දැයි සෙවීමට භාවිතා කරයි.

```

>>> list1=['a','b','a','c','a']
>>> list1.count('a')
3

```

extend()

වෙනත් List එකක ඇති අයිතම දැනට පවතින List එකකට එක් කිරීම සඳහා භාවිතා කරයි.

```

>>> list1=['a','b','a','c','a']
>>> list2=[1,2,3]
>>> list2.extend(list1)
>>> print(list2)
[1, 2, 3, 'a', 'b', 'a', 'c', 'a']

```

insert()

List එකක සුළු අංකය අනුව අයිතම එක් කිරීම සඳහා භාවිතා කරයි.

```

>>> list2=[1,2,3]
>>> list2.insert(0,5)
>>> print(list2)
[5, 1, 2, 3]

```

pop()

List එකක ඇති අවසාන අයිතමය හෝ ලබාදුන් සුළු අංකයට අදාළ අයිතමය ඉවත් කිරීම සඳහා භාවිතා කරයි.

```

>>> list1=['a','b','a','c','a']
>>> list2=[1,2,3]
>>> list1.pop()

```

```
'a'
>>> list2.pop(1)
2
>>> print(list1)
['a', 'b', 'a', 'c']
>>> print(list2)
[1, 3]
```

remove()

නෝරාගත් අයිතමයක් List එකකින් ඉවත් කිරීම සඳහා භාවිතා කරයි.

```
>>> animals=['cat','dog','rat','bat']
>>> animals.remove('dog')
>>> print(animals)
['cat', 'rat', 'bat']
```

reverse()

List එකක අයිතම දකුණේ සිට වම් දැක්වීම සඳහා භාවිතා කරයි.

```
>>> list1=[10,2,5,15,12,1,7]
>>> list1.reverse()
>>> print(list1)
[7, 1, 12, 15, 5, 2, 10]
```

sort()

List එකක ඇති අයිතම ආරෝහන හෝ අවරෝහන පිළිවෙලට සකස් කිරීම සඳහා භාවිතා කරයි.

ආරෝහන පිළිවෙලට සකස් කිරීම.

```
>>> list1=[10,2,5,15,12,1,7]
>>> list1.sort()
>>> print(list1)
[1, 2, 5, 7, 10, 12, 15]
```

අවරෝහන පිළිවෙලට සකස් කිරීම.

```
>>> list1=[10,2,5,15,12,1,7]
>>> list1.sort(reverse=True)
>>> print(list1)
[15, 12, 10, 7, 5, 2, 1]
```

Tuple

මෙම වර්ගයේ දත්තයක් කොමා වලින් වෙන්කරනු ලබන පටිපාටිගත අගයන්ගෙන් සමන්විතවේ. Tuple දත්ත ව්‍යුහයක් සෑදීම සඳහා () භාවිතා කරයි. මෙය immutable හා iterable දත්ත වර්ගයට අයත් වේ. Tuple එකක් නිර්මාණය කළ පසු එහි අඩංගු දත්තයන් කිසිවිටක වෙනස් කළ නොහැක. tuple එකක දත්ත අයිතම list එකකදී මෙන් පරිශීලනය කළ හැක.

```
>>> school=('rce','tmv','gmw','pmv')
>>> animal=('cat','dog','bat','fish')
>>> num=(5,3,2,1,4)
>>> print(num)
(5, 3, 2, 1, 4)
```

```

>>> print(school)
('rcc', 'tmv', 'gmv', 'pmv')
>>> print(school[0])
rcc
>>> print(school[2])
('rcc', 'tmv')
>>> print(school[2:])
('gmv', 'pmv')
>>> print(school[1:3])
('tmv', 'gmv')
>>> print(school[-1])
pmv
>>> print(len(school))
4
>>> print(animals+num)
('cat', 'dog', 'bat', 'fish', 5, 3, 2, 1, 4)
>>> print(5 in num)
True
>>> print(school.index('gmv'))
2
>>> num=(5,3,2,1,4,3,1,3,3)
>>> print(num.count(3))
4

```

Dictionary

Dictionary වර්ගයේ දත්තයක් කොමා වලින් වෙන් කරනු ලබන අතර මේ සඳහා { } වරහන් භාවිතා කරයි. පටිපාටිගත නොවන යතුරු-අගය(Key:Value) යුගල කුලකයකින් යුක්ත වේ. මෙය mutable දත්ත වර්ගයට අයත් වේ. මෙහිදී යතුරු ලෙස අගයක් හෝ අත්පර භාවිතා කළ හැක. Dictionary තුළ ඇති දත්තයන් වෙනස් කළ හැක.

```

>>> dict1={ 1:'aruna',2:'kamal',3:'saman'}
>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> dict3={ 1:'sun',2:'mon',3:'tue'}
>>> print(dict1)
{ 1: 'aruna', 2: 'kamal', 3: 'saman'}
>>> print(dict2)
{'a': 'bimal', 'b': 'tamal', 'c': 'nimal'}
>>> print(dict3)
{ 1: 'sun', 2: 'mon', 3: 'tue'}

```

එකම key එක දෙවරක් භාවිතා කළ විට පෙර අගය මත පසුව එක් කරනු ලබන අගය මත ලියවීම සිදුවේ.

```

>>> animals={ 1:'cat',2:'dog',3:'bat',1:'fish'}
>>> print(animals)
{ 1: 'fish', 2: 'dog', 3: 'bat'}

>>> animals={ 1:'cat',2:'dog',3:'bat',1:'fish'}
>>> print(animals[2])
Dog
>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> print(dict2['a'])

```

Bimal

Dictionary වල වටිනාකම් යාවත්කාලීන කිරීම

```
>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> dict2['a']='sunil'
>>> dict2['c']='tharidu'
>>> print(dict2)
{'a': 'sunil', 'b': 'tamal', 'c': 'tharidu'}

>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> print(dict2.items())
dict_items([('a', 'bimal'), ('b', 'tamal'), ('c', 'nimal')])

>>> print(dict2.keys())
dict_keys(['a', 'b', 'c'])
>>> print(dict2.values())
dict_values(['bimal', 'tamal', 'nimal'])

>>> dict3=dict1.copy()
>>> print(dict3)
{1: 'aruna', 2: 'kamal', 3: 'saman'}
```

Dictionary වල සුළු නොමැති අතර යතුරු පමණක් ඇත.

Dictionary තුළ ඇති එක් එක් යතුරු ඉවත් කිරීම.

```
>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> del dict2['a']
>>> print(dict2)
{'b': 'tamal', 'c': 'nimal'}
```

Dictionary තුළ ඇති සියලු දත්ත ඉවත් කිරීම.

```
>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> dict2.clear()
>>> print(dict2)
{}
```

Dictionary සම්පූර්ණයෙන්ම ඉවත් කිරීම.

```
>>> dict2={'a':'bimal','b':'tamal','c':'nimal'}
>>> del dict2
>>> print(dict2)
```

Set

පයිතන් භාෂාවේ භාවිතා වන තවත් දත්ත වර්ගයකි. මෙය පටිපාටිගත නොවන දත්ත පුරුපයක් වන අතර එකම දත්තය නැවත නැවත භාවිතා කළ නොහැක. මෙහි ඇති ප්‍රධාන වාසිය නම් අවශ්‍ය දත්තයක් පහසුවෙන් සොයාගැනීමට හැකිවීම හා එකම දත්තය නැවත නැවත තිබීම වළකා තිබීමයි. set දැක්වීම සඳහා { } භාවිතා කරයි. set දත්ත වර්ගය ප්‍රධාන ආකාර දෙකකි.

1. set (mutable)
2. Frozenset (immutable)

```
>>> x=[5,6,7,5,9,8]
>>> y=set(x)
>>> print(y)
{5, 6, 7, 8, 9}
```

X නැමැති list එකෙහි 5 ඉලක්කම් දෙකක් ඇත. නමුත් එය set එකක් බවට පත් කළ පසු අනුපිටපත් ඉවත්වී ඇති බව පෙනේ. නව Set () වර්ගය මගින් නිරූපනය වේ.

```
>>> fruits=['mango','banana','apple','mango']
>>> x=set(fruits)
>>> x
{'apple', 'banana', 'mango'}
```

```
>>> animals=['cat','dog','bat','rat','fish','cat']
>>> print(animals)
{'fish', 'cat', 'bat', 'rat', 'dog'}
>>> 'cat' in animals
True
>>> 'lion' in animals
False
>>> 'lion' not in animals
True
>>> 'lion' is animals
False
>>> 'lion' is not animals
True
```

සාමාන්‍ය ගණිතයේදී භාවිතා කරනු ලබන කුලක (set) වල භාවිතා කරන ගණිතකර්ම මෙහිදීද සිදුකරගත හැක.

1. Union
2. Intersection
3. Difference
4. Symmetric difference

```
>>> x=('aruna')
>>> y=('waruna')
>>> x=set('aruna')
>>> y=set('waruna')
>>> x
{'a', 'r', 'n', 'u'}
>>> y
{'r', 'w', 'a', 'n', 'u'}
```

```
>>> y-x # letters in y but not in x
{'w'}
```

```
>>> x|y #letters in either x or y
```

```
['a', 'r', 'w', 'n', 'u']
```

```
>>> x&y # letters in both x and y
['a', 'r', 'n', 'u']
```

```
>>> x^y # letters in x or y but not both
['w']
```

පයිතන් භාෂාවේ භාවිතා වන ගණිත කාරක (Python Basic Operators)

1. අංක ගණිතමය කාරක (Arithmetic Operators)
2. සංසන්දනාත්මක කාරක (Comparison Operators)
3. තාර්කික කාරක (Logical Operators)
4. අයදුම කාරක (Assignment Operators)
5. පිටු අනුකූල කාරක (Bitwise Operators)

අංක ගණිතමය කාරක (Arithmetic Operators)

කාරකය	විස්තරය	නිදසුන (x=5, y=2 නම්)
+	එකතු කිරීම	x+y=7
-	අඩු කිරීම	x-y=3
*	ගුණ කිරීම	x*y=10
/	බෙදීම	x/y=2.5
%	පත්‍රිකය (mod)	x%y=1
**	බලය	x**y=25
//	පූර්ණ සංඛ්‍යාවකට බෙදීම (Floor Division)	x//y=2

```
a = 21
b = 10
c = 0
c = a + b
print ("Value of c is ", c)
c = a - b
print ("Value of c is ", c)
c = a * b
print ("Value of c is ", c)
c = a / b
print ("Value of c is ", c)
c = a % b
print ("Value of c is ", c)
a = 2
b = 3
c = a**b
print ("Value of c is ", c)
a = 10
b = 5
c = a//b
print ("Value of c is ", c)
```

Value of c is 31
Value of c is 11
Value of c is 210
Value of c is 2.1
Value of c is 1
Value of c is 8
Value of c is 2

සංසන්දනාත්මක කාරක (Comparison Operators)

අගයන් දෙකක් හෝ කිහිපයක් සංසන්ධනය කර ප්‍රතිදානය True හෝ False ලෙස ලබාදෙයි.

කාරකය	විස්තරය	නිදසුන ($x=5, y=2$ නම්)
<code>==</code>	සමානයි	$x=y \rightarrow \text{False}$
<code>!=</code>	අසමානයි	$x!=y \rightarrow \text{True}$
<code>></code>	විශාලයි	$x>y \rightarrow \text{True}$
<code><</code>	කුඩයි	$x<y \rightarrow \text{False}$
<code>>=</code>	විශාලයි හෝ සමානයි	$x>=y \rightarrow \text{True}$
<code><=</code>	බලය	$x<=y \rightarrow \text{False}$

```

a = 21
b = 10
c = 0
if (a == b):
    print ("a is equal to b")
else:
    print ("a is not equal to b")
if (a != b):
    print ("a is not equal to b")
else:
    print ("a is equal to b")
if (a < b):
    print ("a is less than b")
else:
    print ("a is not less than b")
if (a > b):
    print ("a is greater than b")
else:
    print ("a is not greater than b")
a = 5;
b = 20;
if (a <= b):
    print ("a is either less than or equal to b")
else:
    print ("a is neither less than nor equal to b")
if (b >= a):
    print ("b is either greater than or equal to b")
else:
    print ("b is neither greater than nor equal to b")

```

භෞතික කාරක (Logical Operators)

and, or සහ not යන භෞතික මෙහෙයවන භාවිතා වන කාරක මෙයට අයත්වේ.

කාරකය	විස්තරය	නිදසුන ($x=5, y=2$ නම්)
and	නාවික and	$x=y$ and $x>y \rightarrow \text{false}$ $x!=y$ and $x>y \rightarrow \text{true}$
or	නාවික or	$x!=y$ or $x<y \rightarrow \text{true}$ $x>y$ or $x!=y \rightarrow \text{true}$
not	නාවික not	$\text{not}(x>y) \rightarrow \text{false}$ $\text{not}(x!=y) \rightarrow \text{true}$

```

a = 10
b = 20
c = 0
if (a and b):
    print ("a and b are true")
else:
    print ("Either a is not true or b is not true")
if (a or b):
    print ("Either a is true or b is true or both are true")
else:
    print ("Neither a is true nor b is true")
a = 0
if (a and b):
    print ("a and b are true")
else:
    print ("Either a is not true or b is not true")
if (a or b):
    print ("Either a is true or b is true or both are true")
else:
    print ("Neither a is true nor b is true")
if not(a and b):
    print ("Either a is not true or b is not true")
else:
    print ("a and b are true")

```

a and b are true
Either a is true or b is true or both are true
Either a is not true or b is not true
Either a is true or b is true or both are true
Either a is not true or b is not true

ආදේශන කාරක (Assignment Operators)

නිසිසම් විචල්‍යයක් සඳහා අගයක් ආදේශ කිරීමේදී යොදාගත හැකි මෙහෙයවනයකි. මෙහිදී ආදේශවන විචල්‍යය operand ලෙස හඳුන්වයි.

කාරකය	විස්තරය	නිදසුන ($x=5, y=2$ නම්)
=	දකුණු පස අගය වම්පස අගයට ආදේශ කිරීම	$y=x \rightarrow y=5$
+=	දකුණු පස ඇති අගය වම්පස අගයට එකතුකර ප්‍රතිඵලය ක්‍රමවත් වම්පසට ආදේශ කිරීම	$x+=y \rightarrow x=7$
-=	වම්පස අගයෙන් දකුණුපස අගය අඩුකර ප්‍රතිඵලය නැවත වම්පසට ආදේශ කිරීම	$x-=y \rightarrow x=3$
=	දකුණු පස අගය වම්පස අගයෙන් ගුණකර ප්‍රතිඵලය වම්පසට ආදේශ කිරීම	$x=y \rightarrow x=10$

y	0	0	0	0	1	0	1	1
xly	0	0	0	0	1	1	1	1
දශමය අගය								15

Binary XOR

x=15 හා y=11 ලෙස ගනමු.

x	0	0	0	0	1	1	1	1
y	0	0	0	0	1	0	1	1
x^y	0	0	0	0	0	1	0	0
දශමය අගය								4

Binary ones complement

පරිගණකවලදී ධන සංඛ්‍යා සෘජුවම භාවිතා කර හැකි අතර සෘණ සංඛ්‍යා නිරූපනය කරනු ලබන්නේ දෙවන අනුපූරක ආකාරයෙනි.

දෙවන අනුපූරක සංඛ්‍යාව = සංඛ්‍යාව +1

සංඛ්‍යාව = 15 → 00001111

දෙවන අනුපූරක සංඛ්‍යාව = 00001111 +1 = 00010000

දශමය සංඛ්‍යාව = -16 (00001111 පළමු අනුපූරකයට හැරවූ විට 11110000 ලෙස ලැබේ. මෙහි Sign පිටුව 1 නිසා සෘණ සංඛ්‍යාවක් ලෙස ලියනු ලබයි.)

වම් පිරුමාරු කිරීම (Left shift)

x=15, y=11 නම්

x	0	0	0	0	1	1	1	1
x<<2	0	0	0	0	0	0	1	1
දශමය අගය								3

y	0	0	0	0	1	0	1	1
y>>2	0	0	0	0	0	0	1	0
දශමය අගය								2

දකුණට පිරුමාරු කිරීම (Right shift)

x=15, y=11 නම්

x	0	0	0	0	1	1	1	1
x>>2	0	0	1	1	1	1	0	0
දශමය අගය								60

y	0	0	0	0	1	0	1	1
y>>2	0	0	1	0	1	1	0	0
දශමය අගය								44


```
a = 60      # 60 = 0011 1100
b = 13      # 13 = 0000 1101
c = 0
c = a & b;   # 12 = 0000 1100
print ("Value of c is ", c)
```

```
Value of c is 12
Value of c is 61
Value of c is 49
Value of c is -61
Value of c is 240
Value of c is 15
```

```
c = a | b;   # 61 = 0011 1101
print ("Value of c is ", c)
```

```
c = a ^ b;   # 49 = 0011 0001
print ("Value of c is ", c)
```

```
c = ~a;      # -61 = 1100 0011
print ("Value of c is ", c)
```

```
c = a << 2;   # 240 = 1111 0000
print ("Value of c is ", c)
```

```
c = a >> 2;   # 15 = 0000 1111
print ("Value of c is ", c)
```

Membership Operators

මෙහිදී in සහ not in කාරක භාවිතා කරයි. යම්කිසි විචල්‍යයක් තුළ අන්තර්ගත දත්තයක් නම් සත්‍යය ලෙසත් අන්තර්ගත නොවන දත්තයක් නම් අසත්‍යය ලෙසත් දැක්වීම සිදුවේ. සත්‍යය බව දැක්වීමට in කාරකයද අසත්‍යය බව දැක්වීමට not in කාරකයද භාවිතා කරයි.

```
>>> a=[1,2,3,5]
>>> print (3 in a)
True
>>> print(4 in a)
False
```

Identity Operators

මෙහිදී is සහ is not කාරක භාවිතා කරයි. යම්කිසි විචල්‍යය දෙකක් සසඳා එක් විචල්‍යයක් තුළ අන්තර්ගත දත්තයක් අනෙක් විචල්‍යය තුළ පවතිනම් සත්‍යය (True) ලෙසත් අන්තර්ගත නොවන දත්තයක් නම් අසත්‍යය (False) ලෙසත් දැක්වීම සිදුවේ. සත්‍යය බව දැක්වීමට is කාරකයද අසත්‍යය බව දැක්වීමට is not කාරකයද භාවිතා කරයි.

```
>>> a=5
>>> b=5
>>> if a is b:
    print('a equal to b')
else:
    print('a not equal to b')
```

Output

a equal to b

```
>>> a=5
>>> b=5
```

```
>>> if a is not b:
    print('a not equal to b')
else:
    print('a equal to b')
```

Output

a equal to b

කාරක ප්‍රමුඛතාවය

පයිතන් ක්‍රමලේඛන භාෂාවේදී පහත සඳහන් කාරක ප්‍රමුඛතා අනුපිළිවෙල භාවිතා කරයි.

ප්‍රමුඛතා මට්ටම	කාරකය	ප්‍රමුඛතාවය වැඩි
1	**	
2	- + ~ (unary plus and minus), complement	
3	* / % //	
4	+ -	
5	>> <<	
6	&	
7	^	
8	<= < > >=	
9	== !=	
10	= += -= *= /= %= //= = &= >>= <<= **=	
11	is, is not	
12	in, not in	ප්‍රමුඛතාවය අඩු
13	not, and, or	

```
>>> 5+2*3
11
>>> 5-2*6
-7
>>> 6+2**3
14
>>> (5+2)//2+2**4
19
>>> 5/2+3
5.5
>>> 4/2+5//2
4.0
```

```
>>> 2+3**2+2**2
15
>>> 8//2**3*2-6//4+1
2
```

Escape characters

අනුලක්ෂණය	විස්තරය
\n	Newline
\\	Backslash (\)
\'	Single quote (')
\''	Double quote (")
\a	ASCII bell
\b	ASCII Backspace
\f	ASCII Form feed
\t	Tab

```
>>> print('Ict\ninstitute')
Ict
institute
>>> print('Ict\tinstitute')
Ict   institute
>>> print('Ict\\institute')
Ict\institute
>>> print('Ict\'institute')
Ict'institute
>>> print('Ict"institute')
Ict"institute
```

පාලන ව්‍යුහ (Control structure)

වරණය (Selection)

If statement

කොන්දේසියක් තෘප්ත වන්නේ නම් පමණක් ප්‍රකාශනය ක්‍රියාත්මක වේ.

```
උදාහරණ: විෂයකට ලබාගත් ලකුණු ඇතුලත් කර එය 50 හෝ 50ට වඩා වැඩි නම් සමත් ලෙස දැක්වීම.
marks=int(input('Enter marks: '))
if marks>=50:
    print('Pass')
```

```
Enter marks: 60
Pass
```

If ...else statement

මෙහිදී කොන්දේසිය තෘප්ත වන්නේ නම් පළමු ප්‍රකාශයද නැතිනම් දෙවන ප්‍රකාශයද ක්‍රියාත්මක වේ.

```
උදාහරණ: විෂයකට ලබාගත් ලකුණු ඇතුලත් කර එය 50 හෝ 50ට වඩා වැඩි නම් සමත් ලෙසත් නැතිනම් අසමත් ලෙස දැක්වීම.
```

```
marks=int(input('Enter marks: '))
if marks>=50:
    print('Pass')
else:
    print('fail')
```

```
Enter marks: 45
fail
```

elif statement

උදාහරණ: විෂයකට ලබාගත් ලකුණු ඇතුළත් කර එය 75 හෝ 75ට වඩා වැඩි නම් 'Excellent' ලෙසත් 50 හෝ 50ට වඩා වැඩි නම් 'Good' ලෙසත් නැතිනම් 'Weak' ලෙස දැක්වීම.

```
marks=int(input('Enter marks: '))
if marks>=75:
    print('Excellent')
elif marks>=50:
    print('Good')
else:
    print('Weak')
```

```
Enter marks: 76
Excellent
```

Nested if Statement

කොන්දේසියකට පසුව කොන්දේසියක් වශයෙන් කොන්දේසි සමූහයක් යෙදෙන අවස්ථාවක භාවිතා කරයි.

උදාහරණ : සංඛ්‍යා තුනකින් විශාලතම සංඛ්‍යාව සෙවීම.

```
n1=int(input('Enter number 1 :'))
n2=int(input('Enter number 2 :'))
n3=int(input('Enter number 3 :'))
if n1>n2:
    if n1>n3:
        print('maximum number :',n1)
    else:
        print('maximum number :',n3)
elif n2>n3:
    print('maximum number :',n2)
else:
    print('maximum number :',n3)
```

```
Enter number 1 :4
Enter number 2 :5
Enter number 3 :7
maximum number : 7
```

පුනර්කරණය (Repetition /Looping)

පුනර්කරණය වන වාර ගණන නිශ්චිතව දන්නා විට ක්‍රමලේඛනයක් ක්‍රියාත්මක කිරීම සඳහා භාවිතා කරයි.

While loop

යම් කොන්දේසියක් සත්‍යව පවතින තුරු වගන්තියක් හෝ වගන්ති කිහිපයක් නැවැත නැවැත ක්‍රියාත්මක වීම සිදුවේ.

උදාහරණ : 1 සිට 5 දක්වා සංඛ්‍යා මුද්‍රණය කිරීම

```
x=1
while x<=5:
    print(x, end=" ")
    x=x+1
```

x=1	1	1 2 3 4 5
while x<=5:	2	
print(x)	3	
x=x+1	4	
	5	

For loop

වගන්තියක් හෝ කිහිපයක් නිෂ්චිත වාර ගණනක් පුනරාවර්තය කිරීමට භාවිතා වේ. මෙය යොදාගත හැක්කේ අවස්ථා දෙකකදී පමණි.

1. යම්කිසි සංඛ්‍යා පරාසයක් තුළ පුනරාවර්තය සඳහා
2. දත්ත ලැයිස්තුවක් භාවිතා කිරීම සඳහා

```
>>> for x in range(5):
    print(x)
```

0
1
2
3
4

```
>>> for x in range(1,5):
    print(x)
```

1
2
3
4

```
>>> for x in range(1,5,2):
    print(x)
```

1
3

```
>>> city='polonnaruwa'
>>> for i in city:
    print(i)
```

p
o
l
o
n
n
a
r
u
w
a

```
>>> x=['a','aruna','ict',5,2]
>>> for i in x:
```



```

        print(i)
a
aruna
ict
5
2

```

Nested loop

පුනරාවර්තන තුළ තවත් පුනරාවර්තන යෙදෙන විට භාවිතා කරයි.

උදාහරණ : 1 සිට 20 දක්වා ඇති ප්‍රථමක සංඛ්‍යා ලබාගැනීම

```

i = 2
while(i < 20):
    2 is prime
    j = 2
    3 is prime
    while(j <= (i/j)):
        5 is prime
        if not(i%j): break
        7 is prime
        j = j + 1
        11 is prime
    13 is prime
    if (j > i/j) :
        17 is prime
        print (i, " is prime")
        19 is prime
    i = i + 1

```

උදාහරණ : 2 සහ 3 ගුණක වල ලබාගැනීම.

```

for i in range (2,4):
    print (i, 'multiplication table')
    print ('-----')
    for x in range(1,13):
        print(i,'*',x,'=',i*x)

```

2 multiplication table

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20
2 * 11 = 22
2 * 12 = 24
```

3 multiplication table

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
3 * 11 = 33
3 * 12 = 36
```

ප්‍රතිකර්මයක් තුළ වරණයක් යෙදීම

උදාහරණ : 1 සිට 15 දක්වා ඇති සිත්තේ සහ ඉරට්ටේ සංඛ්‍යා වෙන වෙනම ප්‍රතිදානය කිරීම.

```
for i in range(16):
    if i%2==0:
        print(i,'is an even number')
    else:
        print(i,'is an odd number')
```

```
0 is an even number
1 is an odd number
2 is an even number
3 is an odd number
4 is an even number
5 is an odd number
6 is an even number
7 is an odd number
8 is an even number
9 is an odd number
10 is an even number
11 is an odd number
12 is an even number
13 is an odd number
14 is an even number
15 is an odd number
```

පුනර්කරණ පාලන වගන්ති (Loop control statements)

Break Statement

පුනර්කරණයක් ක්‍රියාත්මක වෙමින් පවතින අවස්ථාවක අතරමගින් පුනර්කරණය නතර කර ගැනීම සඳහා යොදාගත හැක.

උදාහරණ : 1 සිට 5 දක්වා සංඛ්‍යා මුද්‍රණය වන ක්‍රමලේඛයක 3වන සංඛ්‍යාවේදී ක්‍රමලේඛය නතර කිරීම.

x=1

```
while x<=5:    1
    if x==3:    2
        break
    print(x)
    x=x+1
```

උදාහරණ : polonnaruwa යන වචනයේ n අකුරේදී ක්‍රමලේඛනය නතර කිරීම.

```
for i in 'polonnaruwa':    P
    if i=='n':              o
        break              1
    print(i)                o
```

Continue Statement

පුනර්කරණයක සමහර ප්‍රකාශන අතහැර ඊළඟ ප්‍රකාශනයට යාම සිදුවේ.

උදාහරණ : 1 සිට 5 දක්වා සංඛ්‍යා මුද්‍රණය වන ක්‍රමලේඛයක 3වන සංඛ්‍යාව අතහැර ක්‍රමලේඛය ක්‍රියාත්මක කිරීම.

```
for i in range(1,6):
    if i==3:    1
        continue    2
    print(i)    4
                5
```

උදාහරණ : polonnaruwa යන වචනයේ n අකුරු අතහැර ඊළඟ අකුරු මුද්‍රණය කිරීම.

```
for i in 'polonnaruwa':    P
    if i=='n':              o
        continue           1
    print(i)                o
                            a
                            r
                            u
                            w
                            a
```

Pass Statement

පුනර්කරණයක් ක්‍රියාත්මකවීමේදී අතරමැදි යම්කිසි වාක්‍යයක් එක් කළ යුතු නම් මෙය භාවිතා කරයි.

උදාහරණ : 1 සිට 5 දක්වා සංඛ්‍යා මුද්‍රණය වන ක්‍රමලේඛයක 3වන සංඛ්‍යාවට පෙර ict institute ලෙස එක්තරා ක්‍රමලේඛය ක්‍රියාත්මක කිරීම.

```

for i in range(1,6):
    if i==3:
        pass
        print('ict institute')
    print(i)

```

පයිතන් ශ්‍රිත (Python function)

ශ්‍රිතයක් යනු අවශ්‍ය කාර්යයක් හෝ කාර්යයන් කිහිපයක් කරගැනීම සඳහා භාවිතා කරනු ලබන ප්‍රකාශ අනුක්‍රමයකි. ශ්‍රිතයක අදාළ කාර්යය හෝ කාර්යයන් ශ්‍රිතයේ නිර්වචනය මගින් දැක්වේ. ශ්‍රිත මගින් ක්‍රමලේඛයක ඇති කොටසක් තනි ඒකකයක් ලෙස උපුටාගෙන එම කොටස එම ක්‍රමලේඛයේම වෙනත් ක්‍රමලේඛයක නැවත නැවත භාවිතයට ගතහැකි පරිදි පහසුකම් ලබාදේ. එමගින් ක්‍රමලේඛයක් තුළ එකම කේතය නැවත නැවත ලිවීමට අවශ්‍යය නොවේ. ශ්‍රිත ප්‍රධාන අතාර දෙකකි.බ.

1. කලින් සකස් කරන ලද ශ්‍රිත (Standard Function)
2. පරිච්ඡේදයට අවශ්‍යය පරිදි සකසන ලද ශ්‍රිත (User Define Function)

විචල්‍යයන් හා පරාමිතින් අතර වෙනස (Difference between variable and parameters)

විචල්‍යයන් යනු මතකයේ දත්තයක් ගබඩා කර තබාගන්නා ස්ථානයක් වන අතර එම දත්ත ක්‍රියාපටිපාටියකට භාග්‍යම සඳහා පරාමිතින් උදව් වේ.

පරාමිති වර්ග

ආදාන පරාමිති (Input Parameters)

ආදාන හෝ ප්‍රතිදාන පරාමිතින් (Input/output parameters)

ශ්‍රිතයක් නිර්වචනය කිරීම (Define a function)

පයිතන් භාෂාවේදී ශ්‍රිතයක් නිර්වචනය කිරීම සඳහා 'def' යන මූලපදය භාවිතා කරයි. විධිමත් පරාමිති ලැයිස්තුවක් වරහන් තුළ ඇතුළත් කරනු ලබන අතර (:) සංකේතය මගින් අවසන් කරයි. ශ්‍රිතයේ නම හඳුන්වනයක් බැවින් හඳුන්වන නාමා ලිවීමේදී භාවිතා කරනු ලබන නීති මේ සඳහා වලංගු වේ. පරාමිති ලැයිස්තුව පරාමිති එකක් හෝ කිහිපයක් තිබිය හැක. පරාමිති නොමැති ශ්‍රිත ද සකස් කළ හැක. ශ්‍රිතයක් ක්‍රියාත්මක වන්නේ කැඳවූ විටය. ක්‍රියාවට නංවන ශ්‍රිතය සහ එහි පරාමිතින් මෙහිදී ලබාදිය යුතුය. ශ්‍රිතයකට තනිව ක්‍රියාත්මක විය වෙනාහැකි අතර එය ක්‍රියාත්මක වන්නේ ප්‍රධාන ක්‍රමලේඛයක් තුළය. පහත අකාරයට ශ්‍රිතය අර්ථදැක්විය හැක.

```

def function_name(arg1,arg2):
    statment1
    statment2

```

පරාමිතින් නොමැතිව ශ්‍රිත නොගනුගැනීම

```

def display():
    print()

```

```

#function
def display():
    print('hi')
#program
display()#calling the function
print('Student')
display()#calling the function
print('Aruna')

```

```

hi
Student
hi
Aruna

```

```

print(s,fn[0],sn[0],ln[0],su)
#program
fn=input('Enter first name: ')
sn=input('Enter second name: ')
ln=input('Enter last name: ')
su=input('Enter surname: ')
g=input("Enter 'male' or 'female' :")
c=input("Enter 'married' or 'unmarried' :")
name(fn,sn,ln,su,g,c)

```

උදාහරණ 1

```

def changeme( mylist ):
    "This changes a passed list into this function"
    mylist = [1,2,3,4]; # This would assign new reference in mylist
    print ("Values inside the function: ", mylist)
    return

# Now you can call changeme function
mylist = [10,20,30];
changeme( mylist );
print ("Values outside the function: ", mylist)

```

Output

Values inside the function: [1, 2, 3, 4]
Values outside the function: [10, 20, 30]

උදාහරණ 2

```

# Function definition is here
def printinfo( name, age ):
    "This prints a passed info into this function"
    print ("Name: ", name)
    print ("Age :", age)
    return;

# Now you can call printinfo function
printinfo( age=38, name="aruna" );

```

Output

Name: aruna
Age : 38

උදාහරණ 3

```

# Function definition is here
def printinfo( name, age = 17 ):
    "This prints a passed info into this function"
    print ("Name: ", name)
    print ("Age :", age)

```


return statement

ත්‍රිතයක් කැඳවන සෑම අවස්ථාවකම අගයක් මුද්‍රාණය. එය return යන පයිතන් මූලපදය මගින් ප්‍රධාන ක්‍රමලේඛයට අගයක් මුද්‍රාණයීම සිදුකළ හැකිය. මුද්‍රාණය වන ලද වගන්තියේ ත්‍රිතය තුළ අගයක් නොමැති විට None යන අගය ලබාදේ.

උදාහරණ 1

සංඛ්‍යා දෙකක් ආදානය කර ඒවායේ එකතුව ලබාගැනීමට ක්‍රමලේඛනයක් ත්‍රිත භාවිතයෙන් ලිවීම.

```
#function
def sum(x,y):
    print('Total is :',x+y)
#program
x=int(input('Enter a number 1: '))
y=int(input('Enter a number 2: '))
sum(x,y)
```

```
Enter a number 1: 5
Enter a number 2: 6
Total is : 11
```

```
#function
def sum(x,y):
    return x+y
#program
x=int(input('Enter a number 1: '))
y=int(input('Enter a number 2: '))
print('Total is :',sum(x,y))
```

උදාහරණ 2

පළමු නම, දෙවන නම සහ අවසාන නම ඇතුළත් කර මුලකුරු සමග අවසාන නම දැක්වීම සඳහා පයිතන් ක්‍රමලේඛනයක් ලියන්න.

```
#function
def name(fn,sn,ln):
    print(fn[0],',',sn[0],',',ln)
#program
fn=input('Enter first name: ')
sn=input('Enter second name: ')
ln=input('Enter last name: ')
name(fn,sn,ln)
```

```
Enter first name: Aruna
Enter second name: Kumara
Enter last name: Herath
A . K . Herath
```

උදාහරණ 3

පළමු නම, දෙවන නම, අවසාන නම, වාසගම, ස්ත්‍රී/ පුරුෂ බව සහ විවාහක අවිවාහක බව ඇතුළත් කර Mr, Mrs. හෝ Miss. ඇතුළත් මුලකුරු සමග වාසගම දැක්වීම සඳහා පයිතන් ක්‍රමලේඛනයක් ලියන්න.

```
#function
def name(fn,sn,ln,su,g,c):
    if g=='male':
        s='Mr'
    elif g=='female':
        s='Mrs'
    if c=='married':
        s='Mrs'
    else:
        s='Miss'
```

```
Enter first name: Chanaka
Enter second name: aruna
Enter last name: kumara
Enter surname: herath
Enter 'male' or 'female' :male
Enter 'married' or 'unmarried' :married
Mr . C . a . k . herath
```

```
return;
```

```
# Now you can call printinfo function  
printinfo( age=18, name="kasun" );  
printinfo( name="supun" );
```

Output

```
Name: kasun  
Age : 18  
Name: supun  
Age : 17
```

උදාහරණ 4

```
# Function definition is here  
sum = lambda arg1, arg2: arg1 + arg2;  
  
# Now you can call sum as a function  
print ("Value of total : ", sum( 10, 20 ))  
print ("Value of total : ", sum( 20, 20 ))
```

Output

```
Value of total : 30  
Value of total : 40
```

ශ්‍රිත විස්තරයන් (function arguments)

පයිතන් පවිත්‍රයක භාෂාවේදී ශ්‍රිත විස්තරයන් කිහිපයක් භාවිතා කරයි. ඒවා පයිතන් ශ්‍රිත වල ඇති විශේෂාංග ලෙසද හැඳින්විය හැක.

1. Default arguments
2. Keyword arguments
3. Required arguments
4. Variable arguments

Default arguments

ශ්‍රිත නිරවචනයේදී ස්වයං පැවැරූ අගයන් භාවිතා කළ හැකිය. යම් අගයක් ශ්‍රිතය කැඳවීමේදී නොමැති නම් ශ්‍රිතය තුළ අන්තර්ගත ස්වයං පැවැරූ අගය කැඳවීම සිදුකරයි.

```
#default arguments
```

```
#function
```

```
def myself(name='kamal',age=38,address='kaduruwela'):  
    print('Name is : ',name)  
    print('Age is : ',age)  
    print('Address is : ',address)  
    return  
#program  
myself(name='aruna',address='polonnaruwa')
```

Output

```
Name is : aruna  
Age is : 38  
Address is : polonnaruwa  
Name is : herath  
Age is : 35  
Address is : kaduruwela
```

```
myself(name='herath',age=35)
```

Keyword arguments

මූල පදයක් ශ්‍රිත කැඳවීම සඳහා භාවිතා කරයි නම් එය පරාමිති නාමයන් හඳුනා ගනී.

```
#keyword arguments
```

Output

```
#function
```

```
def myself(str):
```

Aruna herath

```
    print(str)
```

Nethumsa herath

```
    return
```

```
#program
```

```
myself(str='Aruna herath')
```

```
myself(str='Nethumsa herath')
```

Required arguments

ශ්‍රිතයක් තුළ විස්තරයන් භාවිතා කිරීමේදී එය පිළිවෙලකට යැවිය යුතුය. එමෙන්ම ශ්‍රිතය කැඳවීමේ අඩුම තරමින් එක් විස්තරයක් හෝ පැවැරිය යුතුය. එසේ නොමැති නම් දෝෂ පණිවිඩයක් ලබාදෙයි.

```
#keyword arguments
```

Output

```
#function
```

```
def myself(str):
```

Error

```
    print(str='aruna')
```

```
    return
```

```
#program
```

```
myself()
```

Variable length arguments

යම් ශ්‍රිතයක් නිර්වචනයට වඩා වැඩි විස්තරයන් සඳහා ශ්‍රිත සැකසීම සිදුකරයි නම් ඒවා මෙලෙස හඳුන්වයි. විශේෂිත (*) සලකුණක් අදාළ විචල්‍යයට පෙර යොදාගැනේ. එමගින් මූලපද නොවන විචල්‍යයන් සඳහා අගයයන් රඳවා තබාගනී.

```
#Variable length arguments
```

Output

```
#function
```

```
def my_function(*kids):
```

The youngest child is Sandun

```
    print("The youngest child is " + kids[1])
```

```
#program
```

```
my_function("Suranga", "Sandun", "Thusitha")
```

```
#Variable length arguments
```

Output

```
#function
```

```
def my_sum(*args):
```

25

```
    return sum(args)
```

30

```
#program
```

```
print(my_sum(1, 3, 5, 7, 9))
```

```
print(my_sum(2, 4, 6, 8, 10))
```

විචල්‍යයන්ගේ සීමාවන් ඇදීම

පයිතන් භාෂාව විචල්‍යය සඳහා අගයන් ලබාදෙන ස්ථානය එම විචල්‍යයේ සීමාව (scope) තීරණය කරනු ලබයි. ශ්‍රිතයක් තුළ විචල්‍යයන් ප්‍රධාන ආකාර දෙකකට වර්ග කළ හැකිය.

1. ස්ථානීය විචල්‍යය (Local Variable)
2. ගෝලීය විචල්‍යයන් (Global Variable)

ස්ථානීය විචල්‍යය (Local Variable)

ශ්‍රිතය තුළ විචල්‍යයක් ආරම්භ වෙනම් එය ස්ථානීය විචල්‍යයක් ලෙස හැඳින්වේ. මෙම විචල්‍යය භාවිතා කළ හැක්කේ ශ්‍රිතය තුළදී පමණි.

ගෝලීය විචල්‍යයන් (Global Variable)

ගෝලීය විචල්‍යයක් ශ්‍රිතයක් තුළදී හා ඉන් පිටතදී භාවිතා කළ හැකිය. එය ක්‍රමලේඛනයේ ඕනෑම ස්ථානයක භාවිතා කළ හැකිය. ශ්‍රිතයක් තුළදී ගෝලීය විචල්‍යයක් සකස් කර ගැනීම සඳහා ශ්‍රිතය තුළ global ලෙස යොදා විචල්‍යය සකස් කරගත යුතුය.

```
x=5 #global variable
#function
def func1():
    x=3 #local variable
    print(x)
#program
func1()
```

Output

3

```
x=5 #global variable
#function
def func1():
    x=3 #local variable
    print(x)
#program
func1()
print(x)
```

Output

3

5

```
i=5 #global variable
#function
def func1():
    x=3 #local variable
    print(x)
#program
func1()
print(x)
```

Output

3

error

```
i=5 #global variable
#function
def func1():
    global x
    x=3 #global variable
    print(x)
#program
func1()
print(x)
```

Output

3

3

පයිතන් භාෂාවේදී ගොනු හැසිරවීම (File handling in python)

ක්‍රමලේඛනයක් තුළින් ලබාදෙන දත්ත ප්‍රතිදාන ලෙස දෙයාකාරයකට සිදුකළ හැකිය.

1. මුද්‍රණ යන්ත්‍රයක් භාවිතයෙන් මුද්‍රණය කරගැනීම.
2. ගොනු හෝ දත්ත පාදක භාවිතා කර ආවයන උපාංගයක් තුළ ගබඩා කිරීම.

ගොනු සඳහා සිදුකළ හැකි මෙහෙයුම්

1. ගොනුවක ඇති දත්ත කියවීම (Read)
2. ගොනුවක දත්ත තැන්පත් කිරීම (Write)
3. ගොනුවක අවසානයට දත්ත එකතු කිරීම (Append)
4. ගොනුවක ඇති දත්ත යාවත්කාලීන කිරීම (Update)

ගොනු භාවිතයේදී සිදුකළ යුතු ප්‍රධාන කාර්යයන් තුනක් ඇත.

1. ගොනුවට සම්බන්ධතාවයක් විවෘත කිරීම
2. ගොනුවේ ඇති දත්ත කියවීම හෝ ගොනුවට දත්ත එක් කිරීම
3. ගොනුව වැසීම

ගොනුවට සම්බන්ධතාවයක් විවෘත කිරීම

open("file_name", 'mode')

ප්‍රවේශ ආකාර (Access mode)	විස්තරය (Description)
'r'	ගොනුවේ ඇති දත්ත කියවීම පමණයි.
'w'	ගොනුවට දත්ත ලිවීම පමණයි.
'a'	ගොනුවේ අගට දත්ත එක් කිරීම.
'r+'	ගොනුවේ ඇති දත්ත කියවීම සහ ලිවීම.
'w+'	ගොනුවට දත්ත ලිවීම සහ කියවීම.

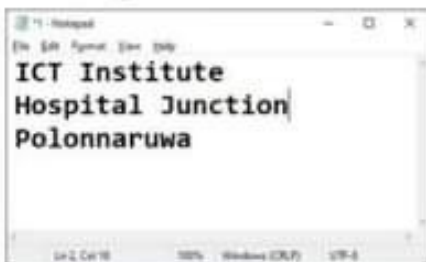
```
x=open('l.txt','r')
x.close()
```

ඉහත පයිතන් ගොනුව සුරකින ලද ස්ථානයේ l.txt ලෙස ගොනුවක් සකස් වී ඇති බව දැකිය හැක.

ගොනුව ඇති සියළුම දත්ත කියවීම.

මේ සඳහා පහත ආකාරයේ ගොනුවක් සකස් කර එය පයිතන් ගොනුව සුරකින ලද ස්ථානයේම සුරකින්න.

l.txt ගොනුව



```
x=open('l.txt','r')
data=x.read()
print(data)
x.close()
```

Output

ICT Institute
Hospital Junction
Polonnaruwa

ගොනුව ඇති අකුරු කිහිපයක් කියවීම.

```
x=open('l.txt','r')
data=x.read(3)
print(data)
x.close()
```

Output

ICT

අකුරු තුනක් පමණක් ප්‍රදර්ශනය වී ඇත.

ගොනුව ඇති එක් පේලියක් පමණක් කියවීම.

Output

ICT Institute

උසස් පෙළ තොරතුරු හා සන්නිවේදන තාක්ෂණ පාඨමාලා ක්‍රියාත්මක වී ඇත.


```
x=open('l.txt','r')
data=x.readline()
print(data)
x.close()
```

පෙළියෙන් පෙළිය ගොනුවක ඇති සියළුම පෙළි නිශ්චිත.

1 ක්‍රමය

```
x=open('l.txt','r')
data=x.readline()
while data:
    print(data)
    data=x.readline()
x.close()
```

Output

```
ICT Institute
Hospital Junction
Polonnaruwa
```

2 ක්‍රමය

```
x=open('l.txt','r')
for i in x:
    print(i)
x.close()
```

Output

```
ICT Institute
Hospital Junction
Polonnaruwa
```

```
x=open('l.txt','r')
data=x.readlines()
print(data)
x.close()
```

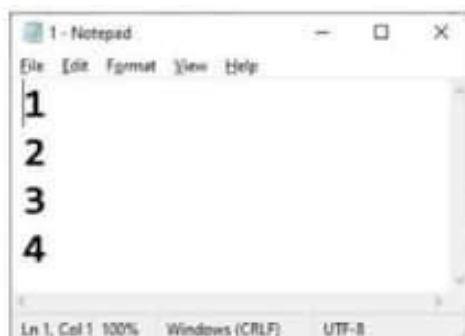
Output

```
['ICT Institute\n', 'Hospital Junction\n', 'Polonnaruwa']
```

ලිස්ට් එකක් තුළ සියළුම පෙළි දැක්වේ.

ඔප්ලව් 1

පහත ගොනුවේ ඇති සංඛ්‍යා නිශ්චය ඒවායේ එකතුව සහ සාමාන්‍යය ලබා ගැනීමට පයිතන් ක්‍රමලේඛණය



```
tot=0
x=open('l.txt','r')
data=x.readlines()
for i in data:
    no=int(i)
    tot=tot+no
avg=tot/3
print('Total is', tot)
print('Average is', round(avg,2))
```

Output

```
Total is 10
Average is 3.33
```

ගොනුවකට දත්ත ලිවීම

```
x=open('a.txt','w')
x.write('Aruna Herath')
x.close()
```



මෙම කේතය ක්‍රියාත්මක කළ විට පයිතන් ගොනුව සූරකින ලද ස්ථානයේ a.txt නැමැති ගොනුවක් නිර්මාණය වේ.

ගැටලුව 1

1 සිට 10 දක්වා ඇති ඔත්තේ සංඛ්‍යා ගොනුවකට ලිවීමට අදාළ python ක්‍රමලේඛනය

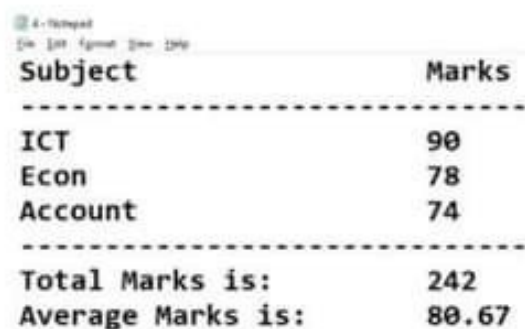
```
x=open('2.txt','w')
for i in range(1,10,2):
    b=str(i)
    x.write(b+'\n')
x.close()
```



ගැටලුව 2

විෂයන් තුනක ලකුණු ඇතුළත් කර ඒවායේ එකතුව සහ සාමාන්‍යය සෙවීම සොයා ගොනුවක ලිවීමට පයිතන් ක්‍රමලේඛනය.

```
total=0
x=open('4.txt','w')
x.write('Subject\t\t\t\t\tMarks\n')
x.write('-----\n')
for i in range(3):
    name=str(input('Enter Subject:'))
    marks=str(input('Enter Marks:'))
    x.write(name+'\t\t\t\t'+marks+'\n')
    total=total+int(marks)
average=round(total/3,2)
t=str(total)
a=str(average)
x.write('-----\n')
x.write('Total Marks is: \t'+t+'\n')
x.write('Average Marks is: \t'+a+'\n')
x.close()
```

A screenshot of a Notepad window titled '4 - Notepad'. The menu bar includes File, Edit, Format, View, and Help. The content is a table with two columns: Subject and Marks. The subjects listed are ICT, Econ, and Account. The marks are 90, 78, and 74 respectively. Below the table, the total marks and average marks are calculated and displayed.

Subject	Marks
ICT	90
Econ	78
Account	74
Total Marks is:	242
Average Marks is:	80.67

ඇවලව 3

```
for count in range(2):
    subject=['ICT','BS','AC']
    total=0
    x=open('5.txt','a')
    name=str(input('Enter your name:'))
    x.write('-----\n')
    x.write('Your name :'+name+'\n')
    x.write('-----\n')
    x.write('Subject\t\t\t\t\tMarks\t\t\tGrade\n')
    x.write('-----\n')
    for i in range(3):
        s=subject[i]
        marks=str(input('Enter '+s+' Marks:'))
        m=int(marks)
        if m>=75:
            g='A'
        elif m>=65:
            g='B'
        elif m>=55:
            g='C'
        elif m>=35:
            g='S'
        else:
            g='W'
        x.write(s+'\t\t\t\t\t'+marks+'\t\t\t'+g+'\n')
        total=total+int(marks)
    average=round(total/3,2)
    t=str(total)
    a=str(average)
    x.write('-----\n')
    x.write('Total Marks is: \t'+t+'\n')
    x.write('Average Marks is: \t'+a+'\n')
    x.write('-----\n')
    x.close()
```

ගොනුවක නම වෙනස් කිරීම

```
import os
os.rename('1.txt','abc.txt')
```

ගොනුවක් ඉවත් කිරීම

```
import os
os.remove('a.txt')
```

ෆෝල්ඩරයක් සකස් කිරීම

```
import os
os.mkdir('aruna')
```

File Name:
 Date: / /

Your name :kamal

Your name :451

Subject	Marks	Grade
ICT	50	S
BS	60	C
AC	70	B
Total Marks is:		180
Average Marks is:		60.0

os විධානය මගින් windows මෙහෙයුම් පද්ධතියේ විධාන ක්‍රියාත්මක කළ හැක.

Notepad මෘදුකාංගය විවෘත කිරීම

```
import os
cmd='notepad'
os.system(cmd)
```

mspaint මෘදුකාංගය විවෘත කිරීම

```
import os
cmd='mspaint'
os.system(cmd)
```

ෆෝල්ඩරයක් ඉවත් කිරීම

```
import os
os.rmdir('aruna')
```

Python Sort

මෙහිදී කිසියම් අයිතම සමූහයක් ආරෝහණ හෝ අවරෝහණ පිළිවෙලකට සකස් කිරීම සිදුකරනු ලබයි. මේ සඳහා භාවිතා වන ප්‍රධාන ක්‍රම දෙකකි.

1. වරණ තේරීම (Selection sort)
2. බුබුළු තේරීම (Bubble sort)

වරණ තේරීම (Selection sort)

මෙහිදී ලැයිස්තුවක ඇති අයිතම පිළිවෙලකට සකසන තුරු තේරීම සිදුකරයි. ලැයිස්තුවේ ඇති අයිතම අතරින් විශාලතම අගය හා කුඩාම අගය නිවැරදිව ස්ථානගත කිරීම මගින් මෙම තේරීම සිදුකරයි.

#function	3, 1, 41, 59, 26, 53, 59
def selection_sort(L):	1, 3, 41, 59, 26, 53, 59
for i in range(len(L)-1):	1, 3, 26, 59, 41, 53, 59
min_index = i	1, 3, 26, 41, 59, 53, 59
for j in range(i+1, len(L)-1):	1, 3, 26, 41, 53, 59, 59
if L[j] < L[min_index]:	
min_index = j	
L[i], L[min_index] = L[min_index], L[i]	
#program	
L = [3, 1, 41, 59, 26, 53, 59]	
print(L)	
selection_sort(L)	
print(L)	

Output

[3, 1, 41, 59, 26, 53, 59]
[1, 3, 26, 41, 53, 59, 59]

බුබුළු තේරීම (Bubble sort)

යම්කිසි ලැයිස්තුවක් පිළිවෙලකට සකසන තුරු ලැයිස්තුවක ඇති යාබද මූලාංග සංසන්ධනය කර ඒවායෙන් කුඩා අගය මුලින් විශාල අගය වලින් පිහිටන පරිදි පිළිවෙලකට තේරීම බුබුළු තේරීම මගින් සිදුකරයි.

#function	
def bubble_sort(our_list):	19, 13, 6, 2, 18, 19
for i in range(len(our_list)-1):	13, 19, 6, 2, 18, 19
for j in range(len(our_list) - 1):	13, 6, 19, 2, 18, 19
if our_list[j] > our_list[j+1]:	13, 6, 2, 19, 18, 19
our_list[j], our_list[j+1] = our_list[j+1], our_list[j]	13, 6, 2, 18, 19, 19
#program	
our_list = [19, 13, 6, 2, 18, 8]	
bubble_sort(our_list)	
print(our_list)	

Output

[2, 6, 8, 13, 18, 19]

6, 2, 13, 18, 19, 19
2, 6, 13, 18, 19, 19

Python ක්‍රමලේඛනයක් My SQL දත්ත සමුදායක් සමඟ සම්බන්ධ කිරීම.

සම්බන්ධතාවය පවත්දැයි පරීක්ෂා කිරීම.

- Python connector නැමැති මෘදුකාංගය ස්ථාපනය කරගන්න.
<https://dev.mysql.com/downloads/connector/python/2.0.html>
- පහත විධානය භාවිතා කර මෘදුකාංගය ස්ථාපනය කර ඇත්දැයි බලන්න.
- `import mysql.connector`
- නැවැත්වූ ස්ථාපනය කර ඇත්නම් දෝෂ පණිවිඩයක් නොපෙන්වයි.

දත්ත සමුදාය සමඟ සම්බන්ධතාවයක් ඇති කර ගැනීමට (Create a connection) my sql ස්ථාපනයේදී ලබාදුන් පරිශීලක නාමය සහ මුරපදය සහිතව පහත පයිතන් කේතය ලියා ක්‍රියාත්මක කරන්න.

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword"
)
print(mydb)
```

දත්ත සමුදායක් සකස් කිරීම

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword"
)
mycursor = mydb.cursor()
mycursor.execute("CREATE DATABASE mydatabase")
```

සකස් කළ දත්ත සමුදායන් නිරීක්ෂණය

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="myusername",
    passwd="mypassword")
mycursor = mydb.cursor()
mycursor.execute("SHOW DATABASES")
for x in mycursor:
    print(x)
```

වගුවක් නිර්මාණය කිරීම

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
```



```

user="yourusername",
passwd="yourpassword",
database="mydatabase"
)
mycursor = mydb.cursor()
mycursor.execute("CREATE TABLE customers (name VARCHAR(255), address
VARCHAR(255))")

```

සකස් කළ වගු නිරීක්ෂණය කිරීම

```

import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()

mycursor.execute("SHOW TABLES")

for x in mycursor:
    print(x)

```

වගුවකට ප්‍රාථමික යතුරක් එක් කිරීම

```

import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
mycursor.execute("CREATE TABLE customers (id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255), address VARCHAR(255))")

```

සකස් කළ වගුවකට ප්‍රාථමික යතුරක් එක් කිරීම

```

import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
mycursor.execute("ALTER TABLE customers ADD COLUMN id INT AUTO_INCREMENT
PRIMARY KEY")

```

වගුවකට දත්ත එක් කිරීම

```
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)

mycursor = mydb.cursor()
sql = "INSERT INTO customers (name, address) VALUES (%s, %s)"
val = ("John", "Highway 21")
mycursor.execute(sql, val)
mydb.commit()
print(mycursor.rowcount, "record inserted.")
```

වගුවකට එකවර රෙකෝඩ් කිහිපයක් එක් කිරීම

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
sql = "INSERT INTO customers (name, address) VALUES (%s, %s)"
val = [
    ('Peter', 'Lowstreet 4'),
    ('Amy', 'Apple st 652'),
    ('Hannah', 'Mountain 21'),
    ('Michael', 'Valley 345'),
    ('Sandy', 'Ocean blvd 2'),
    ('Betty', 'Green Grass 1'),
    ('Richard', 'Sky st 331'),
    ('Susan', 'One way 98'),
    ('Vicky', 'Yellow Garden 2'),
    ('Ben', 'Park Lane 38'),
    ('William', 'Central st 954'),
    ('Chuck', 'Main Road 989'),
    ('Viola', 'Sideway 1633')
]

mycursor.executemany(sql, val)
mydb.commit()
print(mycursor.rowcount, "was inserted.")
```

වගුවක ඇති සියළු දත්ත නිරීක්ෂණය කිරීම

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
mycursor.execute("SELECT * FROM customers")
myresult = mycursor.fetchall()
for x in myresult:
    print(x)
```

නිරූ කිහිපයක ඇති දත්ත ලබාගැනීම

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
mycursor.execute("SELECT name, address FROM customers")
myresult = mycursor.fetchall()
for x in myresult:
    print(x)
```

My SQL Where විධානය භාවිතා කිරීම

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
sql = "SELECT * FROM customers WHERE address = 'Park Lane 38'"
mycursor.execute(sql)
myresult = mycursor.fetchall()
for x in myresult:
    print(x)
```

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
```

```

)
mycursor = mydb.cursor()
sql = "SELECT * FROM customers WHERE address LIKE '%way%'"
mycursor.execute(sql)
myresult = mycursor.fetchall()
for x in myresult:
    print(x)

```

```

import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="mydatabase"
)
mycursor = mydb.cursor()
sql = "SELECT * FROM customers WHERE address = %s"
adr = ("Yellow Garden 2", )

mycursor.execute(sql, adr)
myresult = mycursor.fetchall()
for x in myresult:
    print(x)

```